From Chase to Meta-Chase

Wenfei Fan[1,2,3], Xiaomeng Luo[3], Ping Lu[3]
[1]Shenzhen Institute of Computing Sciences [2]University of Edinburgh [3]Beihang University
wenfei@inf.ed.ac.uk, {luping, luoxm}@buaa.edu.cn

***Abstract*—Fishing Fort (FF) implements graph reasoning based on Graph Association Rules (GARs) via chase, supporting explainable inference over graph data with convergence guarantees under fixed rule sets. However, decision-aware graph analytics increasingly require adaptive workflows that dynamically select rule subsets based on task context and runtime constraints. In response to the need, we introduce Agentized Fishing Fort (AFF), a policy-driven multi-agent extension of FF that enables workload-aware deduction through dynamic GAR selection. AFF gives rise to the Meta-Chase problem: logical deduction under rule subsets dynamically determined by policy. Unlike classical chase, Meta-Chase breaks global Church-Rosser guarantees and may yield schedule-dependent outputs. We formalize Meta-Chase, and show that it terminates under all fair schedules. To enforce output-level consistency, we propose three complementary mechanisms for workload-driven deduction to admit a unique output, settle their complexity, and develop effective algorithms. Experiments on real-world graph tasks verify that AFF improves execution effectiveness and efficiency while preserving logical correctness.**

I. Introduction

Fishing Fort (FF) [1], [2] is a graph reasoning framework with *Graph Association Rules* (GARs) [3], which integrate graph pattern matching with logic and machine-learning (ML) predicates. By applying GARs via chase [4], FF provides explainable inference with convergence guarantees under fixed rule sets. It has proven successful in decision-critical domains, including drug discovery [5], battery manufacturing [6], and online recommendation [7], supporting tasks such as candidate selection, defect diagnosis, recommendation and optimization, with factual [8] and counterfactual [9] explanations.

However, modern graph analytics increasingly operate under *dynamic workloads*, in which reasoning tasks evolve over time and vary across task contexts. In many real-world deployments, different analytical tasks require different subsets of GARs, depending on runtime conditions such as task objectives, data availability, or computational constraints, among other things. Executing a fixed static rule set for all tasks may incur unnecessary computation and long response time.

Example 1: Consider a battery manufacturing pipeline in which performance degradation is detected in various production batches. Root-cause analysis may involve multiple diagnostic tasks, including safety inspection, recovery assessment, and temperature fluctuation analysis (see [6]). Each task depends on a different combination of GARs that capture causal relationships among production parameters and environmental factors. Furthermore, multiple diagnostic agents may operate concurrently, each focusing on task-specific reasoning. This requires the reasoning system to adapt dynamically to workloads and tasks, rather than running a fixed rule set.

FF executes a fixed GAR set Σ via chase [4] on graph G:

$$(G', \Gamma^*) = \mathsf{Chase}(G, \Sigma, \Gamma_0),$$

where chase repeatedly applies applicable GARs in Σ on graph G until no new facts can be derived. Here Γ_0 contains initial validated facts over the input graph G, and Γ^* is the resulting logical closure. While this supports comprehensive inference, executing the full GAR set via the chase for every task incurs unnecessary computation and delays decision-making in dynamic workloads that require task-specific reasoning. □

The challenge is not just efficiency. Classical chase assumes a fixed GAR set Σ, under which the closure is uniquely determined by (G, Σ, Γ_0) and is independent of rule application order by the Church-Rosser property, *i.e.,* all valid rule application orders yield the same terminal closure [10]. In workload-driven settings, however, different tasks may activate different GAR subsets. This raises a key question: how should deduction be defined when the rule set itself becomes dynamic?

AFF. To address this challenge, we introduce *Agentized Fishing Fort* (AFF), a policy-driven extension of FF that enables adaptive GAR-based deduction through dynamic rule selection. AFF organizes GARs into reusable execution units, called *motifs*, which are selected based on task context and runtime constraints. Rather than executing the full GAR set, AFF activates only task-relevant motifs and supports multi-agent coordination via shared derived facts. Based on execution outcomes, AFF continuously updates its policy for motif selection, *i.e.,* deduction strategies to adapt to evolving workloads.

This elevates policy from an execution-level optimization mechanism to a semantic component of deduction itself. Reasoning in AFF is therefore performed over dynamically selected GAR subsets rather than a fixed global rule set. At runtime, AFF incrementally applies selected GARs over validated facts to derive task-specific closures.

However, this transformation fundamentally changes the semantics of deduction. Classical chase operates over a fixed GAR set Σ, under which the closure is uniquely determined by (G, Σ, Γ_0). In contrast, AFF performs deduction over dynamically selected rule subsets $\Sigma_{W_t} \subseteq \Sigma$, which may vary across tasks and runtime states. As a consequence, the derived closure may depend on policy decisions and execution schedules, and Church-Rosser no longer holds globally.

Contributions & Organization. This paper introduces Agentized Fishing Fort (AFF), the first formal framework for workload-driven GAR-based deduction under dynamic rule selection, and establishes a foundation for policy-driven rea-

soning via Meta Chase. After reviewing Fishing Fort (FF) in Section II, we make the following contributions.

(1) AFF framework (Section III). We introduce AFF, a policy-driven multi-agent extension of FF that supports workload-aware deduction via dynamic selection of GAR subsets. AFF organizes GARs into reusable motifs and enables agents to coordinate inference by sharing derived facts across tasks. Policies are continuously updated from execution outcomes, for deduction strategies to adapt to evolving workloads.

(2) Meta Chase formalization (Section IV). We formalize the Meta Chase problem, which conducts logical deduction under dynamically selected GAR subsets determined by policy. Thus unlike classical chase, Meta Chase may yield schedule-dependent outputs. We show that Meta Chase terminates under all fair schedules, and produces task-specific closures in polynomially many chasing steps in data complexity. We also show that deciding whether Meta Chase admits a unique terminal output under policy-driven rule selection is coNP-hard.

(3) Confluence mechanisms for Meta Chase (Section V). To address the loss of global confluence, we develop three mechanisms for enforcing output-level consistency under policy-driven deduction: (a) a sufficient output-commutativity condition for restoring Church-Rosser behavior; (b) a canonicalization framework for schedule-robust outputs; and (c) a two-phase protocol that separates evidence derivation from decision generation. We establish complexity bounds, *e.g.,* $\mathsf{P}^{\mathsf{NP}}_{||}$-complete (data complexity) and Π^p_2-complete (combined complexity) for deciding output-commutativity, and NP-complete for optimal canonicalization (data complexity); this said we develop effective algorithms with performance guarantees.

(4) Empirical validation. Using real-world graph query loads, we show that (a) AFF has high decision quality; its recall reaches 0.94 under limited execution budgets. (b) AFF substantially improves efficiency over classical chase, up to $2.69\times$ faster than FF by applying only task-relevant GAR subsets. (c) Adaptive policy learning consistently outperforms static and random schedules, showing the benefit of state-aware task selection. (d) Our canonicalization and two-phase mechanisms achieve output-level consistency with negligible overhead (about 1% and $< 0.01\%$, respectively). Together, these verify that workload-aware deduction substantially cuts reasoning cost while preserving decision quality and robustness.

We discuss related work in Section VII and the novelty of this work in Section VIII. Detailed proofs are deferred to [11].

II. Fishing Fort and Chase

This section reviews the logical reasoning foundation of Fishing Fort (FF), which performs inference over graph-structured data via *Graph Association Rules* (GARs). We first present GARs, followed by the workflow of FF, and finally review chase-based deduction and its convergence guarantees.

A. Graph Association Rules

Let $G = (V, E, \mathcal{L}, \ell, \mathcal{A}, \lambda)$ be a *labeled property graph with attributes*, where: (a) V is a finite set of vertices; (b) $E \subseteq V \times \mathcal{L} \times V$ is a finite set of labeled directed edges; (c) $\mathcal{L}$ is a set of labels; (d) $\ell : V \to \mathcal{L}$ assigns labels to vertices; (e) $\mathcal{A}$ is a set of vertex attributes; and (f) λ assigns attribute values. Attributes are schema-free and may vary across vertices.

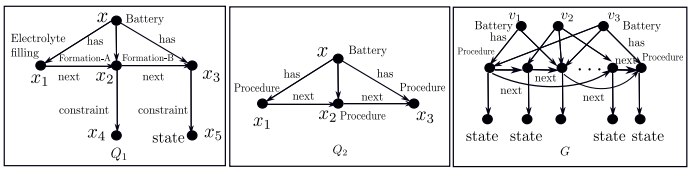


Fig. 1: The pattern in Example 2

Graph patterns. A *graph pattern* is $Q[\bar{x}] = (V_Q, E_Q, L_Q)$, where (1) V_Q (resp. E_Q) is a finite set of *pattern nodes* (resp. *edges*) as above; (2) $L_Q : V_Q \to \mathcal{L}$ assigns a label $L_Q(u)$ to each pattern node; and (3) $\bar{x}$ is a list of distinct variables, each uniquely denoting a node in V_Q.

A *match* of pattern $Q[\bar{x}]$ in graph G is a homomorphism h from Q to G such that (a) for each node $u \in V_Q$, $L_Q(u) = \ell(h(u))$; and (b) for each pattern edge $e = (u, l, u')$ in Q, $e' = (h(u), l, h(u'))$ is an edge in G.

Predicates. A *predicate p of pattern $Q[\bar{x}]$* is defined as

$$p \ ::= \ \ x.A \oplus y.B \mid x.A \oplus c \mid l(x, y) \mid M(x, y),$$

where x, y are variables in $\bar{x}$ denoting vertices, $x.A$ is the value of attribute A of pattern node x, $\oplus \in \{=, \neq, \geq, >, \leq, <\}$ is a comparison operator, $l(x, y)$ denotes the existence of an edge (x, l, y) from vertex x to y, c is a constant, and M is a pretrained ML model that returns a Boolean value, *e.g.,* $M_{\mathsf{er}}(x, y)$ for entity resolution, $M_{\mathsf{lp}}(x, y)$ for link prediction, $M_{\mathsf{anom}}(x)$ for anomaly detection based on production signals, and $M_{\mathsf{cap}}(x)$ for capacity prediction in battery grading. We refer to $l(x, y)$ as edge predicates, $x.A \oplus y.B$ and $x.A \oplus c$ as *logic predicates*, and $M(x, y)$ as an *ML predicate*.

GARs. A *Graph Association Rule* (GAR) is defined as [3]:

$$\varphi = Q[\bar{x}](X \to p_0),$$

where $Q[\bar{x}]$ is a graph pattern; X is a conjunction of predicates of $Q[\bar{x}]$; p_0 is predicate (a fact) of $Q[\bar{x}]$ to be derived. Here X and p_0 are *precondition* and *consequence* of φ, respectively.

Semantics. Let Γ be a set of validated facts. Given a GAR $\varphi = Q[\bar{x}](X \to p_0)$ and a match h of $Q[\bar{x}]$ in G, we say that X *holds* under h *w.r.t.* Γ, denoted by $\Gamma \models h(X)$, if for each predicate $p \in X$: (a) if p is a comparison predicate $x.A \oplus y.B$ (resp. $x.A \oplus c$), then $h(x).A \oplus h(y).B$ (resp. $h(x).A \oplus c$) holds in Γ; (b) if p is an edge predicate $l(x, y)$, then $(h(x), l, h(y)) \in \Gamma$; and (c) if p is an ML predicate $M(x, y)$, then $M(h(x), h(y)) = \mathsf{true}$. When $\Gamma \models h(X)$, φ derives fact $h(p_0)$, written as $\Gamma \vdash_\varphi h(p_0)$, which extends Γ with $h(p_0)$.

Each GAR is interpreted as a universally quantified implication over all matches of its pattern in G, and derives only facts over existing graph objects without introducing fresh objects.

Example 2: Consider GARs of [6] for battery manufacturing, defined using the graph patterns Q_1 and Q_2 shown in Figure 1.

(1) A capacity grading GAR is $\varphi_1 = Q_1[x](X_1 \rightarrow x.\mathsf{capacity} = 8)$, where X_1 requires that the matched formation procedures of x satisfy prescribed ranges on charging current and voltage attributes. More specifically, together with pattern Q_1, precondition X_1 specifies the following: (1) the weight before and after the Electrolyte Filling procedure (x_1) is 555 ± 25g and 605 ± 25g, respectively; (2) its Formation-A procedure (x_2) uses a constant charging current at 3.8A with initial voltage between 0–100mV, and it is constrained by a final state 324 (x_4); and (3) its Formation-B procedure (x_3) uses a constant charging current at 8.8A, with initial voltage between 3.3–3.4V constrained by final state 738 (x_5). Intuitively, GAR φ_1 assigns capacity grades based on conditions X_1, providing an interpretable rule for cell evaluation.

(2) An anomaly detection GAR is $\varphi_2 = Q_2[x](X_2 \rightarrow x.\mathsf{status} = \mathsf{anomalous})$, where X_2 is the conjunction $x.\mathsf{voltage} \leq 1.9V \wedge x.\mathsf{temperature} \geq 86°C \wedge M_a(x) = \mathsf{false}$. Here $M_a(x)$ denotes the output of a pretrained anomaly detection model. The purpose of φ_2 is to incorporate domain knowledge into complement data-driven predictions, thus improving ML-based detection by reducing false negatives.

(3) A GAR is $\varphi_3 = Q_2[x](x.\mathsf{status} = \mathsf{anomalous} \rightarrow x.\mathsf{action} = \mathsf{requiresInspection})$, which derives an inspection flag from the detected anomaly. Unlike φ_2, which improves ML prediction accuracy, φ_3 supports downstream decision-making by providing explanations for detected faults. □

B. Fishing Fort: An Overview

Fishing Fort (FF) performs logical deduction over graph-structured data using a set of GARs [1], [2]. Formally, FF takes as input a property graph G, a GAR set Σ, and an initial set of validated facts Γ_0. Its output is Γ^*, which is the logical closure of Γ_0 under Σ. More specifically, it is the smallest fact set such that (a) $\Gamma_0 \subseteq \Gamma^*$, and (b) for any GAR $\varphi = Q[\bar{x}](X \rightarrow p_0) \in \Sigma$ and any match h of $Q[\bar{x}]$ in G, if $\Gamma^* \models h(X)$, then $h(p_0) \in \Gamma^*$. Intuitively, Γ^* contains all facts that can be derived from Γ_0 by repeatedly applying GARs in Σ.

Specifically, FF computes Γ^* by extending the chase [4].

Chasing step. Given a fact set Γ, a *chasing step* applies a GAR $\varphi \in \Sigma$ with a match h of $Q[\bar{x}]$ such that $\Gamma \models h(X)$ and $h(p_0) \notin \Gamma$, and produces a new fact set $\Gamma' = \Gamma \cup \{h(p_0)\}$. A step is *invalid* if $\Gamma \not\models h(X)$ or $h(p_0) \in \Gamma$, and *valid* otherwise.

Chasing sequence. A chasing sequence starting from Γ_0 is a sequence $\Gamma_0 \rightarrow \Gamma_1 \rightarrow \cdots \rightarrow \Gamma_k$, where each transition is a valid chasing step. It is *terminal* if no valid step exists at Γ_k. Such a chasing sequence produces a logical closure Γ^*.

Church-Rosser property. Under a fixed GAR set Σ, chase is confluent: all terminal valid chasing sequences starting from Γ_0 yield the same closure Γ^* [3]. Hence, the output Γ^* of FF is uniquely determined by (G, Σ, Γ_0), denoted by $\mathsf{Chase}(G, \Sigma, \Gamma_0)$. Moreover, if the initial fact set Γ_0 and rule set Σ are correct, then the logical closure Γ^* is also correct. This ensures that logical deduction in FF is deterministic and sound; *i.e.,* despite different rule application orders, FF guarantees to converge to a unique final inference results and preserves the correctness of inferred facts.

Example 3: Let the GAR set $\Sigma = \{\varphi_2, \varphi_3\}$ and initial fact set Γ_0 contain measurements of voltage and temperature for a cell c_1. Suppose that $h(x) = c_1$ is a match of $Q_2[x]$ such that $\Gamma_0 \models h(X_2)$. A valid chasing step applies φ_2 to derive $c_1.\mathsf{status} = \mathsf{anomalous}$, yielding $\Gamma_1 = \Gamma_0 \cup \{c_1.\mathsf{status} = \mathsf{anomalous}\}$. Next, φ_3 can be applied to Γ_1 with the same match, producing $c_1.\mathsf{action} = \mathsf{requiresInspection}$. Hence, $\Gamma_2 = \Gamma_1 \cup \{c_1.\mathsf{action} = \mathsf{requiresInspection}\}$. Since no further valid chasing step exists, the sequence $\Gamma_0 \rightarrow \Gamma_1 \rightarrow \Gamma_2$ is terminal, and Γ_2 is the logical closure Γ^*. Intuitively, Γ_2 not only detects the anomaly of cell c_1 but also derives inspection requirement, returning all actionable consequences that follow from the observed production signals and the GARs in Σ. □

III. Agentized Fishing Fort

This section introduces Agentized Fishing Fort (AFF), a multi-agent extension of Fishing Fort (FF) that enables adaptive deduction under dynamic workloads.

Workflow of AFF. Instead of executing a fixed GAR set Σ, AFF maintains a policy function π that maps a task W and current deduction state Δ to a task-specific GAR subset $\Sigma_W \subseteq \Sigma$. At runtime, an agent executes deduction by applying GARs in Σ_W over an initial fact set Γ_0, producing a task-specific closure Γ^*_W. Different tasks may activate different GAR subsets, and may share derived facts across executions.

Based on execution outcomes, AFF updates policy π to adapt rule selection to evolving workloads.

Motifs and execution order. Building on the GAR semantics in Section [2], AFF introduces *motifs* as execution-level abstractions of GARs. A motif $m = (\varphi, C, S)$ consists of (a) a GAR φ, (b) applicability constraints C over task context, and (c) empirical statistics S recorded from prior executions (*e.g.,* marginal gain, noise rate, runtime). Let $\mathcal{M}$ denote a repository of motifs. For a task W, the policy π selects a subset $\mathcal{M}_W \subseteq \mathcal{M}$, which induces $\Sigma_W = \{\varphi(m) \mid m \in \mathcal{M}_W\}$.

Beyond selecting motifs, AFF also determines their execution order. A workflow $W = \langle m_1, \ldots, m_\ell \rangle$ induces an ordered activation of rule subsets Σ_{W_i}, yielding a deduction sequence $\Gamma_0 \rightarrow \Gamma_1 \rightarrow \cdots$. Different motif orders may produce distinct intermediate states, even though termination under fair schedules is guaranteed (see Theorem 1 in Section IV).

Since deduction is now performed over dynamically selected rule subsets, classical chase semantics no longer suffice. This motivates the Meta-Chase framework developed shortly.

Schedules. Given a workflow $W = \langle m_1, \ldots, m_\ell \rangle$ selected by policy π, a *schedule* $T = \langle \Sigma_{W_1}, \ldots, \Sigma_{W_\ell} \rangle$ is the sequence of rule subsets activated during workload-driven deduction.

A schedule T is *fair* if every task $W \in \mathcal{W}$ whose associated rules remain applicable is eventually activated, as commonly found in practice. We consider fair schedules in the sequel.

FF vs. AFF. FF performs deduction over a fixed GAR set Σ,

yielding a unique closure Γ^*. In contrast, AFF performs deduction over dynamically selected rule subsets Σ_W determined by policy. Consequently, different schedules may yield different outputs, *i.e.,* the Church-Rosser property no longer holds.

Example 4: Consider a battery manufacturing pipeline in which a batch of cells exhibits abnormal discharge behavior during testing. Two tasks may be triggered.

- *Safety inspection* W_1 activates a GAR subset Σ_{risk} that derives $x.\textit{voltage} \leq 1.95V \rightarrow x.\textit{risk} = \textit{thermal}$.
- *Recovery assessment* W_2 activates another subset Σ_{rec} that derives $M_r(x) = \textit{true} \rightarrow x.\textit{recoverable} = \textit{true}$, where M_r is a pretrained recovery predictor.

Both tasks are useful: $\textit{risk}(x)$ supports safety screening, while $\textit{recoverable}(x)$ supports cost-saving rework decisions.

Now consider two decision GARs: $x.\textit{risk} = \textit{thermal} \rightarrow x.\textit{action} = \textit{discard}$, and $x.\textit{recoverable} = \textit{true} \rightarrow x.\textit{action} = \textit{rework}$. Suppose AFF enforces an early-commit protocol: once $x.\textit{action}$ is derived, a fact $x.\textit{locked}$ is added, and the policy π disables rule subsets that would derive alternative actions.

If safety inspection runs first, $x.\textit{action} = \textit{discard}$ is derived, and recovery rules are disabled. If recovery assessment runs first, $x.\textit{action} = \textit{rework}$ is derived instead.

Hence, different task schedules may produce distinct outputs, illustrating how dynamic subset selection in AFF can break the global Church–Rosser guarantee. Thus, dynamic rule selection may change the semantics of deduction itself, as terminal outputs can depend on policy and execution order. □

IV. From Chase to Meta-Chase

While AFF enables task-aware deduction by dynamically selecting GAR subsets, it also introduces new challenges. In particular, logical deduction is no longer performed over a fixed rule set, and thus requires a new formal framework. This section formalizes the Meta Chase problem, which captures GAR-based deduction under dynamically selected rule subsets.

Meta Chase. Given a graph G, a GAR set Σ, an initial fact set Γ_0, a task schedule $T = \langle W_1, \ldots, W_k \rangle$, and a policy π, Meta Chase iteratively computes $\Gamma_i = \mathsf{Chase}(G, \pi(W_i, \Gamma_{i-1}), \Gamma_{i-1})$ for $i = 1, \ldots, k$. Let Γ_T^* denote the resulting terminal fact set. Unlike classical chase, Γ_T^* may depend on both T and π. Meta Chase thus provides a schedule-dependent deduction semantics for workload-driven reasoning.

Challenges. Unlike classical chase, Meta Chase performs deduction over dynamically selected GAR subsets rather than a fixed rule set. This raises three challenges. (1) *Termination.* Task-dependent rule activation may repeatedly trigger interacting GAR subsets, potentially creating cyclic deduction. (2) *Confluence.* The Church-Rosser property no longer holds, since different task schedules may activate different rule subsets and thus produce different closures (see Example 4). (3) *Output consistency.* Even when different schedules derive different fact sets, decision outputs should be stable, especially when tasks share derived facts and influence subsequent deduction. These challenges call for new foundations for termination, confluence and consistency in policy-driven deduction.

Classical chase defines a unique closure over a fixed rule set Σ. Meta Chase, in contrast, defines a family of schedule-dependent closures $\{\Gamma_T^* \mid T \text{ is fair}\}$, induced by dynamically selected rule subsets. The central challenge is thus no longer closure computation, but closure consistency across schedules.

Complexity. We consider two problems for Meta Chase:

- *Termination(MC).* Given $(G, \Sigma, \Gamma_0, W_1, \ldots, W_k, \pi)$, does Meta Chase terminate?
- *Unique-Closure(MC).* Given $(G, \Sigma, \Gamma_0, \mathcal{W}, \pi)$, do all task sequences over a set $\mathcal{W}$ of tasks yield the same final facts?

We first settle Termination(MC) in positive.

Theorem 1: *Termination(MC): Meta Chase terminates for any fair schedule* $\langle W_1, \ldots, W_k \rangle$ *and any policy* π. □

Intuitively, Theorem 1 holds because (a) the input graph G is finite, (b) every GAR in Σ is a universal rule that derives only facts over existing vertices/edges (*i.e.,* no fresh object/value creation), and (c) chase steps are monotone, *i.e.,* each step only adds a new fact to the current fact set. That is, with universal GARs over a finite graph, Meta Chase cannot "run forever" because it never invents new objects; it can only add previously missing facts, and there are finitely many such facts.

Proof sketch: Let $\mathsf{Fact}(G, \Sigma)$ denote the set of all ground facts derivable on G under Σ. By condition (b), every derived fact arises from a pattern match of some GAR in Σ. Since G is finite, only finitely many such matches exist, and thus $\mathsf{Fact}(G, \Sigma)$ is finite. By condition (c), each valid chasing step strictly enlarges the current fact set. Hence at most $|\mathsf{Fact}(G, \Sigma)|$ valid steps can occur. Under any fair schedule, every persistently applicable rule subset is eventually selected. Therefore, Meta Chase can only add finitely many new facts before reaching a fixpoint, and thus terminates after at most $|\mathsf{Fact}(G, \Sigma)|$ valid steps, regardless of the policy π. □

Unfortunately, Unique-Closure(MC) is not guaranteed. Non-uniqueness may arise as illustrated in Example 4 where different task schedules lead to different terminal decisions.

Worse still, Unique-Closure(MC) is intractable. We show that output uniqueness under dynamic rule selection is much harder than classical fixed-rule confluence, even when individual chase executions remain terminating.

Theorem 2: *Unique-Closure(MC) is* coNP*-hard.* □

Proof sketch: Let $I = (G, \Sigma, \Gamma_0, \mathcal{W}, \pi)$ be an instance of Meta Chase, and let $\mathcal{P}_{\mathsf{out}}$ be a designated set of *output predicates*. For a (fair) task schedule T over $\mathcal{W}$, let Γ_T^* denote the terminal fact set produced by executing Meta Chase on I under T. We say that $I \in \mathsf{UniqueClosure}_{\mathsf{out}}(\mathsf{MC})$ iff for all fair schedules T and T' over $\mathcal{W}$, $\Gamma_T^* \cap \mathcal{P}_{\mathsf{out}} = \Gamma_{T'}^* \cap \mathcal{P}_{\mathsf{out}}$.

We prove the coNP-hardness by reduction from UNSAT, which is coNP-complete (cf. [12]). Given a 3CNF formula Φ, we construct an instance $I_\Phi = (G_\Phi, \Sigma_\Phi, \Gamma_0, \mathcal{W}, \pi)$ such that Φ is unsatisfiable iff $I_\Phi \in \mathsf{UniqueClosure}_{\mathsf{out}}(\mathsf{MC})$, *i.e.,* all task sequences yield the same final fact set (see [11] for details). □

V. Confluence under Meta Chase

Meta Chase may yield non-unique terminal outputs on P_{out} due to policy-driven selection of GAR subsets. This section presents three mechanisms to enforce output-level confluence.

A. Sufficient Condition for Church-Rosser

Non-uniqueness arises when different task-specific GAR subsets derive outputs that influence each other, so that applying one subset first changes the preconditions for the other. Intuitively, if the outputs derived by one subset never affect the applicability of rules in another, then the order in which these subsets are applied should not change the final outputs. This motivates a commutativity condition on GAR subsets.

Sufficient condition. Let $\mathcal{S} = \{\Sigma_W \mid W \in \mathcal{W}\}$ be the collection of rule subsets associated with the tasks in the workload. We say that $\mathcal{S}$ is *output-commutative* if for all $\Sigma_i, \Sigma_j \in \mathcal{S}$ and all fact sets Γ, $\mathsf{Chase}(G, \Sigma_i, \mathsf{Chase}(G, \Sigma_j, \Gamma)) \cap P_{\mathsf{out}} = \mathsf{Chase}(G, \Sigma_j, \mathsf{Chase}(G, \Sigma_i, \Gamma)) \cap P_{\mathsf{out}}$. Here P_{out} is the set of output predicates whose instantiated facts constitute the final decisions produced by deduction. Intuitively, this condition ensures that each subset produces output facts independently of the order in which other subsets are applied.

Theorem 3: *If $\mathcal{S}$ is output-commutative, then Meta Chase is Church-Rosser w.r.t. P_{out} under any fair schedule.* □

Proof sketch: If $\mathcal{S}$ is output-commutative, then any two adjacent applications of GAR subsets can be swapped without affecting the resulting output facts. By repeatedly swapping adjacent steps, any two fair schedules can be transformed into one another; thus their terminal closures agree on P_{out}. □

Example 5: Let Σ_{risk} derive $x.\mathsf{risk} = \mathsf{true}$ from voltage signals, and Σ_{rec} derive $x.\mathsf{recoverable} = \mathsf{true}$ via ML predictions. If none derives $x.\mathsf{action}$, their outputs do not affect each other, and the subsets are output-commutative. However, if Σ_{risk} also derives $x.\mathsf{action}$, then applying Σ_{risk} first may prevent Σ_{rec} from affecting the final decision, breaking commutativity. □

Unfortunately, the condition is nontrivial to check.

Theorem 4: *Given $\mathcal{S}$, deciding whether $\mathcal{S}$ is output-commutative is Π_2^p-complete (combined complexity), and is $\mathsf{P}_{||}^{\mathsf{NP}}$-complete when Γ is fixed (data complexity).* □

In the polynomial hierarchy, (a) Σ_2^p denotes the class of problems that can be solved in NP with access to an NP oracle (*i.e.,* $\Sigma_2^p = \mathsf{NP}^{\mathsf{NP}}$), and Π_2^p is the class of problems solvable in coNP with an NP oracle (*i.e.,* $\Pi_2^p = \mathsf{coNP}^{\mathsf{NP}}$) [13]; both are strictly harder than NP and coNP unless NP = P; (b) $\mathsf{P}_{||}^{\mathsf{NP}}$ is the set of problems that can be determined by a deterministic polynomial-time (PTIME) Turing machine making polynomial many queries to an oracle in parallel, in which each query is answered without knowing the result of other queries [14].

Proof sketch: (1) *Combined complexity*. For the upper bound, consider the complement problem. A witness consists of a fact set Γ, two rule subsets $\Sigma_1, \Sigma_2 \in \mathcal{S}$, and an output predicate $p_0 \in P_{\mathsf{out}}$ such that $p_0 \in \mathsf{Chase}(G, \Sigma_1, \mathsf{Chase}(G, \Sigma_2, \Gamma))$ but $p_0 \notin \mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma))$. Membership of p_0 in the first closure can be checked in NP by guessing a chasing sequence. Non-membership in the second closure is thus in coNP. Hence the existence of such a witness is decidable in Σ_2^p. Thus, deciding whether $\mathcal{S}$ is output-commutative is in Π_2^p.

For the lower bound, we reduce from $\mathrm{QBF}_{\forall\exists}$, which is Π_2^p-complete (cf. [13]). Given a formula $\forall \bar{x}\, \exists \bar{y}\, \Psi(\bar{x}, \bar{y})$, we construct a graph G and rule subsets in $\mathcal{S}$ such that output-commutativity holds iff the formula is true. The construction encodes assignments to $\bar{x}$ as fact sets and uses pattern-matching gadgets to simulate the universal verification over $\bar{y}$. Hence output-commutativity is Π_2^p-hard.

(2) *Data complexity*. We show that the problem is in $\mathsf{P}_{||}^{\mathsf{NP}}$. Consider the membership problem: given $f \in P_{\mathsf{out}}$, determine whether $f \in \mathsf{Chase}(G, \Sigma_i, \mathsf{Chase}(G, \Sigma_j, \Gamma))$. This can be decided using an NP oracle for GAR verification [3]. Hence, a violation of output-commutativity can be checked by testing whether for some $f \in P_{\mathsf{out}}$, $f \in \mathsf{Chase}(G, \Sigma_i, \mathsf{Chase}(G, \Sigma_j, \Gamma))$ but $f \notin \mathsf{Chase}(G, \Sigma_j, \mathsf{Chase}(G, \Sigma_i, \Gamma))$. Since there are only polynomially many candidate output predicates (Theorem 1), all such membership tests can be issued in parallel to an NP oracle. Thus checking non-commutativity is in $\mathsf{P}_{||}^{\mathsf{NP}}$, and so is deciding output-commutativity.

For the lower bound, we reduce from the odd max true 3SAT (OMT3) problem, which is $\mathsf{P}_{||}^{\mathsf{NP}}$-complete [15], [14]. OMT3 is to decide, given a 3SAT formula Φ with variables $\bar{x}$, if $\mu_{\max}(\bar{x})$ is the satisfying truth-assignment with maximum true among all assignments, whether $\mu_{\max}(\bar{x})$ sets an odd number of variables true. Given a 3SAT formula Φ, we construct G, $\mathcal{S}$ and Γ such that output-commutativity holds for $\mathcal{S}$ iff $\mu_{\max}(\bar{x})$ sets an odd number of variables to true. The construction encodes assignments of Φ in G, uses $\mathcal{S} = \{\Sigma_1, \Sigma_2\}$ to simulate formula evaluation, and derives a distinguished predicate p_0 iff the parity condition holds. Hence the problem is $\mathsf{P}_{||}^{\mathsf{NP}}$-hard. □

Algorithm. Despite Theorem 4, we next develop an algorithm for checking whether a collection $\mathcal{S}$ is output-commutative.

By definition, output-commutativity requires that for all $\Sigma_i, \Sigma_j \in \mathcal{S}$ and all fact sets Γ, the two compositions of chase produce identical facts in P_{out}. A naive approach would enumerate all Γ, which is infeasible for large-scale graphs. Our algorithm avoids this by identifying a finite set of *critical fact sets* that suffice to witness divergence on output predicates.

Critical fact sets. Observe that only facts that may enable rules deriving predicates in P_{out} can affect the final outputs. Hence, if non-commutativity exists, there must be a counterexample that consists solely of instantiated preconditions of rules whose heads lie in P_{out}. We thus construct a collection $\mathcal{C}$ of such critical fact sets by instantiating the bodies of rules in $\Sigma^{\mathsf{out}} = \{\varphi \in \mathcal{S} \mid \mathsf{head}(\varphi) \in P_{\mathsf{out}}\}$ over matches of their patterns in G.

For each GAR set $\Sigma \in \mathcal{S}$, the critical fact set $\mathcal{C}_\Sigma$ consists of tuples $(h(X), h(p_0))$ such that there exist a GAR $\varphi = Q[\bar{x}](X \to p_0)$ in Σ and a match of h of Q in G. Different tuples $(h_1(X_1), p), \ldots, (h_n(X_n), p)$ in $\mathcal{C}_\Sigma$ may share predicates, and we adopt the trie structures to store $\mathcal{C}_\Sigma$ [16].

Intuitively, a trie is a tree-based data structure, where an edge denotes a predicate, and a node v_n in the tree represents a subset of predicates appearing along the path from the root to node v_n. Using the trie data structure, we compactly store $\mathcal{C}_\Sigma$.

Witness pairs. To check output-commutativity, we define a set $\mathcal{P}_\Sigma$ of *witness pairs* from the critical fact set $\mathcal{C}_\Sigma$. A pair (p_1, p_0) belongs to $\mathcal{P}_\Sigma$ if (1) $p_0, p_1 \in P_{\mathsf{out}}$; (2) $\mathcal{C}_\Sigma$ contains a tuple (X, p_0) with $p_1 \in X$; and (3) $(X \setminus \{p_1\}, p_0) \notin \mathcal{C}_\Sigma$.

Intuitively, (p_1, p_0) indicates that p_1 may be necessary for deriving p_0. Hence, if Σ_2 derives p_1 before Σ_1 is applied, then Σ_1 may derive p_0; however, the reverse order may fail to derive p_1 and thus p_0. Witness pairs thus capture precisely the dependencies that can lead to violations of output-commutativity.

We can efficiently verify whether $(p_0, p_1) \in \mathcal{P}_\Sigma$, since $X \setminus \{p_1\}$ is a parent node of the one representing X.

Output-commutativity testing. Based on this, we check the output-commutativity of a collection $\mathcal{S}$ of rule subsets as follows. For each witness pair $(p_1, p_0) \in \mathcal{P}_{\Sigma_1}$, let $(X, p_0) \in \mathcal{C}_{\Sigma_1}$ satisfy that $p_1 \in X$ and $(X \setminus \{p_1\}, p_0) \notin \mathcal{C}_{\Sigma_1}$. We then check whether there exists $(X_1, p_1) \in \mathcal{C}_{\Sigma_2}$ such that p_0 cannot be derived from $(X \cup X_1) \setminus \{p_1\}$ using $\Sigma_1 \cup \Sigma_2$. If so, (p_1, p_0) witnesses a violation of output-commutativity, since when $\Gamma = (X_1 \cup X_2) \setminus \{p_1\}$, $p_0 \in \mathsf{Chase}(G, \Sigma_1, \mathsf{Chase}(G, \Sigma_2, \Gamma))$ and $p_0 \notin \mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma))$.

Algorithm. We present Algorithm CheckCommunity in Figure 2. It first computes the critical fact set $\mathcal{C}_\Sigma$ for each GAR set $\Sigma \in \mathcal{S}$ (line 1). Next, it constructs the set $\mathcal{P}_\Sigma$ of witness pairs for each $\Sigma \in \mathcal{S}$ (line 2). Then, it checks whether Σ_1 and Σ_2 are output-communitive for each Σ_1 and Σ_2 in $\mathcal{S}$ (lines 3-9). More specifically, for each pair (p_0, p_1) in $\mathcal{P}_\Sigma$, it first collects all $(X, p_0) \in \mathcal{C}_{\Sigma_1}$ such that $p_1 \in X$, but $(X \setminus \{p_1\}, p_0) \notin \mathcal{C}_{\Sigma_1}$ (line 4), and collects GAR sets Σ_2 that can deduce p_1, *i.e.,* $(X_1, p_1) \in \mathcal{C}_{\Sigma_2}$ (line 5). For each $(X, p_0) \in \mathcal{C}_{\Sigma_1}$ and each $(X_1, p_1) \in \mathcal{C}_{\Sigma_2}$, it first checks whether p_0 *cannot be deduced* from $(X \cup X_1) \setminus \{p_1\}$ using either Σ or Σ_1 (lines 6-7). If so, return false; otherwise, it checks other sets in T and GAR sets in $\mathcal{S}_{p_1}$. After inspecting all pairs in $\mathcal{P}_{\Sigma_1}$ and all $\Sigma_1 \in \mathcal{S}$, it returns true (line 9), *i.e.,* $\mathcal{S}$ is output-communitive.

Example 6: Assume that $\mathcal{S} = \{\Sigma_1, \Sigma_2\}$, where $\Sigma_1 = \{\varphi_2\}$, $\Sigma_2 = \{\varphi_3\}$, and φ_2 and φ_3 are GARs defined in Example 2. Given graph G in Figure 1, CheckCommunity checks whether $\mathcal{S}$ is output-commutative as follows.

(1) It first constructs a critical fact set $\mathcal{C}_{\Sigma_i}$ for each $\Sigma_i \in \mathcal{S}$ ($i \in [1, 2]$), *i.e.,* $\mathcal{C}_{\Sigma_1} = \{(v_j.\mathsf{voltage} \leq 1.9V \wedge v_j.\mathsf{temperature} \geq 86°C \wedge M_a(v_j) = \mathsf{false}, v_i.\mathsf{status}{=}\mathsf{anomalous}) \mid j \in [1, 3]\}$, and $\mathcal{C}_{\Sigma_2} = \{(v_j.\mathsf{status} = \mathsf{anomalous}, v_j.\mathsf{action} = \mathsf{requiresInspection}) \mid j \in [1, 3]\}$. Both $\mathcal{C}_{\Sigma_1}$ and $\mathcal{C}_{\Sigma_2}$ have three elements, one for each battery in graph G.

(2) Next CheckCommunity computes the set of witness pairs for both Σ_1 and Σ_2. For the GAR set Σ_1, $\mathcal{P}_{\Sigma_1}$ contains the following pairs: (a) $(v_j.\mathsf{voltage} \leq 1.9V, v_j.\mathsf{status} = \mathsf{anomalous})$, (b) $(v_j.\mathsf{temperature} \geq 86°C, v_j.\mathsf{status}{=}\mathsf{anomalous})$ and (c) $(M_a(v){=}\mathsf{false}, v.\mathsf{status}, v.\mathsf{status}{=}\mathsf{anomalous})$ with $j \in [1, 3]$, one for each condition in the precondition of φ_2, since φ_2 cannot deduce consequence $v_j.\mathsf{status}{=}\mathsf{anomalous}$ after removing any predicate from precondition. Similarly, $\mathcal{P}_{\Sigma_2} = (v_j.\mathsf{status} = \mathsf{anomalous}, v_j.\mathsf{action} = \mathsf{requiresInspection})$ ($j \in [1, 3]$).

Input: A graph G and a set of tasks $\mathcal{S} = \{\Sigma_W \mid W \in \mathcal{W}\}$.
Output: Whether $\mathcal{S}$ is output-commutative.
1. compute the critical fact sets $\mathcal{C}_\Sigma$ for each GAR set Σ;
2. identify the set $\mathcal{P}_\Sigma$ of witness pairs (p_0, p_1) for Σ;
3. **for each** $\Sigma_1 \in \mathcal{S}$ and $(p_1, p_0) \in \mathcal{P}_{\Sigma_1}$ **do**
4. $T := \{(X, p_0) \in \mathcal{C}_{\Sigma_1} \mid p_1 \in X \wedge (X \setminus \{p_1\}, p_0) \notin \mathcal{C}_{\Sigma_1}\}$;
5. let $\mathcal{S}_{p_1}$ consist of all GAR set $\Sigma_2 \in \mathcal{S}$ deducing p_1;
6. **for each** $(X, p_0) \in T$ and $(X_1, p_1) \in \mathcal{C}_{\Sigma_2}$ with $\Sigma_2 \in \mathcal{S}_{p_1}$ **do**
7. bDeduce := Deduce $(p_0, p_1, X, X_1, \mathcal{C})$;
8. **if** bDeduce = false **then return** false;
9. **return** true;

Fig. 2: Algorithm CheckCommunity

(3) After that, CheckCommunity checks whether $\mathcal{S}$ is output-commutative by inspecting witness pairs in $\mathcal{P}_{\Sigma_1}$ and $\mathcal{P}_{\Sigma_2}$. Consider witness pair $(v_j.\mathsf{status} = \mathsf{anomalous}, v_j.\mathsf{action} = \mathsf{requiresInspection})$ in $\mathcal{P}_{\Sigma_2}$. The set $\mathcal{S}_{p_1}$ consists the GAR set Σ_1, since $\mathcal{C}_{\Sigma_1} = \{(v_j.\mathsf{voltage} \leq 1.9V \wedge v_j.\mathsf{temperature} \geq 86°C \wedge M_a(v_j) = \mathsf{false}, v_j.\mathsf{status} = \mathsf{anomalous}) \mid j \in [1, 3]\}$, which deduces the precondition $v_j.\mathsf{status}{=}\mathsf{anomalous}$ in $\mathcal{P}_{\Sigma_2}$.

We verify that $\mathcal{S}$ is not output-commutative. Let $\Gamma_0 = \{v_j.\mathsf{voltage} \leq \{v_j 1.9V, v_j.\mathsf{temperature} \geq 86°C, M_a(v_j) = \mathsf{false} \mid j \in [1, 3]\}$ using predicates in $\mathcal{P}_{\Sigma_1}$. Then $\mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma_0)) = \{v_j.\mathsf{status} = \mathsf{anomalous}, v_j.\mathsf{action} = \mathsf{requiresInspection} \mid j \in [1, 3]\}$, while $\mathsf{Chase}(G, \Sigma_1, \mathsf{Chase}(G, \Sigma_2, \Gamma_0)) = \{v_j.\mathsf{status} = \mathsf{anomalous} \mid j \in [1, 3]\}$. That is, the two sets are different, and $\mathcal{S}$ is not output-commutative. □

Complexity. CheckCommunity takes $O(|G|^4 f^2(G, \mathcal{S}))$ time, where $f(G, \mathcal{S})$ denotes the cost of enumerating all GAR matches in G. Critical fact sets and witness pairs are computed in $O(f(G, \mathcal{S}))$ time. The dominant cost comes from checking output-commutativity: there are $O(|G|^2)$ witness pairs and $O(f(G, \mathcal{S}))$ critical fact sets, and each Deduce call takes $O(|G|^2 f(G, \mathcal{S}))$ time. Thus it takes $O(|G|^4 f^2(G, \mathcal{S}))$ overall.

Properties. The approach is sound: if a discrepancy on P_{out} is found, then output-commutativity does not hold. Moreover, it is complete: whenever output-commutativity fails, there exists a critical fact set $\Gamma \in \mathcal{C}$ witnessing the failure. When rule patterns are fixed, the number of critical fact sets is polynomial in $|G|$, and each chase invocation over a fixed rule set runs in PTIME. This does not contradict Theorem 4, since it is a sound test, not a complete procedure for the general problem.

B. Canonicalization for Output Uniqueness

When output-commutativity fails, different Meta Chase schedules may yield different outputs in P_{out}. Rather than restoring global commutativity, we enforce *output-level uniqueness* through canonicalization. For each output key, competing facts are treated as candidates and resolved by a deterministic selection rule (*e.g.,* priority, score, or dominance). Consequently, the published outputs depend only on the derived fact set, not on the deduction order.

Theoretical background. Specifically, let Γ^* denote the terminal fact set obtained by Meta Chase under an arbitrary fair schedule. Define a canonicalization function $\mathsf{can}: 2^{\mathsf{Facts}} \to 2^{P_{\mathsf{out}}}$ such that $\mathsf{can}(\Gamma^*) \subseteq \Gamma^* \cap P_{\mathsf{out}}$. We require that function can is (a) *deterministic*, (b) *monotone w.r.t.* set inclusion on non-output facts, and (c) *local* to P_{out}, *i.e.,* it depends only on the attributes and provenance of facts in $\Gamma^* \cap P_{\mathsf{out}}$. Typical instances include: selecting, for each entity x, the output fact with maximum confidence score, or the minimal element under a fixed total order $\prec$. Then the *published output*

$$\mathsf{Out}(\Gamma^*) \;:=\; \mathsf{can}(\Gamma^*)$$

is invariant across schedules, since any two terminal sets Γ^*_T and $\Gamma^*_{T'}$ satisfy $\mathsf{can}(\Gamma^*_T) = \mathsf{can}(\Gamma^*_{T'})$ whenever the canonicalization criterion is schedule-independent.

Example 7: Revisit Example 4, where different task schedules may derive either $x.action = \textit{discard}$ or $x.action = \textit{rework}$ for the same cell x. Let $P_{\mathsf{out}} = \{\textit{action}(\cdot)\}$. Suppose each derived action fact carries a confidence score or priority level, *e.g.,* $x.action = \textit{discard}$ with score 0.92 and $x.action = \textit{rework}$ with score 0.87. Under canonicalization, we define a deterministic selection rule: for each entity x, retain only the action fact with the highest score (or highest priority under a fixed total order). Formally, $\mathsf{can}(\Gamma^*)$ selects, for each x,

$$\arg\max_{a} \; \textit{score}(x.action = a).$$

Even if two schedules T and T' produce different fact sets Γ^*_T and $\Gamma^*_{T'}$, the canonicalized outputs $\mathsf{can}(\Gamma^*_T)$ and $\mathsf{can}(\Gamma^*_{T'})$ coincide if the scoring function depends only on the derived facts and not on rule application order. Thus output-level uniqueness is restored without enforcing global commutativity. □

Complexity. In general, computing a globally optimal canonical set (*e.g.,* maximizing a weighted utility subject to mutual-exclusion constraints among outputs) is NP-hard. Consider the decision problem, denoted by OptCanon: given (a) a set O of output facts (*e.g.,* $O = \Gamma^* \cap P_{\mathsf{out}}$), (b) a symmetric *conflict* relation $\mathsf{Conf} \subseteq O \times O$ indicating which outputs cannot be published together, (c) a nonnegative weight function $w : O \to \mathbb{N}$, and (d) an integer K, decide whether there exists a subset $O' \subseteq O$ such that (i) O' is conflict-free, *i.e.,* for all $o \neq o'$ in O', $(o, o') \notin \mathsf{Conf}$, and (ii) $\sum_{o \in O'} w(o) \geq K$.

Theorem 5: *Problem* OptCanon *is* NP*-complete, and hence computing an optimal canonical output set (maximizing* $\sum_{o \in O'} w(o)$ *subject to conflicts) is* NP*-hard. The hardness holds already in data complexity.* □

Proof sketch: For the upper bound, a certificate is a subset $O' \subseteq O$. We can verify in time polynomial in $|O|$ that O' is conflict-free by checking all pairs in O' against Conf, and that its total weight is at least K by summation. Hence OptCanon is in NP. For the lower bound, we reduce from the independent set problem, which is NP-complete (cf. [12]). The reduction uses only output candidates and their pairwise conflicts, and does not depend on rule patterns or predicates, the hardness holds even when Σ and P_{out} are fixed. Thus OptCanon is NP-complete in data complexity (see [11] for details). □

Input: A set O of candidate output facts (*e.g.,* $O = \Gamma^* \cap P_{\mathsf{out}}$).
Output: A canonicalized output Γ^*.
1. collect the set $C_{x.A}$ of predicates p that contains $x.A$ in P_{out};
2. **for each** $C_{x.A}$ with $x.A$ in P_{out} **do**
3. $\quad \mathsf{Out}(\Gamma^*_{x.A}) := \mathsf{can}(C_{x.A})$;
4. $\Gamma^* := \bigcup_{x.A \in P_{\mathsf{out}}} \Gamma^*_{x.A}$;
5. **return** Γ^*;

Fig. 3: Algorithm Canon

Algorithm. Given a terminal fact set Γ^* produced by Meta Chase, we restrict attention to $O = \Gamma^* \cap P_{\mathsf{out}}$. The algorithm is presented in Figure 3. It first groups output facts by entity (or decision key; line 1), and within each group applies a fixed selection rule to retain a canonical representative, discarding dominated alternatives (lines 2-3). This requires only sorting or local comparison over O, and thus runs in $O(|O| \log |O|)$ time. The published set $\mathsf{Out}(\Gamma^*)$ is unique for any schedule.

Note that this procedure does not solve the general Opt-Canon problem of maximizing a weighted conflict-free subset (Theorem 5), which is NP-hard; instead, it implements a restricted deterministic selection rule (*e.g.,* per-entity max or fixed priority), and is therefore polynomial-time.

Example 8: Continuing with Example 7. Let $P_{\mathsf{out}} = \{\textit{action}(\cdot)\}$ and $O = \{x.action{=}\textit{discard}, x.action{=}\textit{rework}\}$. Assume that the confidence scores of predicates $x.action = \textit{discard}$ and $x.action{=}\textit{rework}$ are 0.92 and 0.87, respectively. We define the following selection rule: for each entity x, retain only the action fact with the highest score.

Algorithm Canon first collects predicates of the form $\textit{action}(\cdot)$ in O, *i.e.,* $x.action = \textit{discard}$ and $x.action = \textit{rework}$, and picks the one with the highest confidence as output, *i.e.,* $x.action = \textit{discard}$. Then $\mathsf{can}(\Gamma^*){=}\{x.action = \textit{discard}\}$. □

Performance guarantees and complexity. The canonicalization layer is *schedule-robust*: for any two fair schedules T, T',

$$\mathsf{Out}(\Gamma^*_T) = \mathsf{Out}(\Gamma^*_{T'}).$$

Moreover, it preserves logical soundness: all published outputs are derived facts in Γ^*. Algorithm Canon runs in $O(|O| \log |O|)$ time, where $O = \mathsf{Out}(\Gamma^*)$ denotes the set of candidate output facts. Since $O \subseteq \Gamma^*$ and $|\Gamma^*|$ is polynomially bounded in $|G|$ (Theorem 1), Canon runs in PTIME in data complexity. Consequently, the overhead of canonicalization is typically negligible compared with the chase itself.

C. Two-Phase Protocol: Derive First, Decide Later

Non-uniqueness in Meta Chase often stems from premature decision derivation. If output predicates in P_{out} are produced before all supporting evidence is available, different schedules may commit to different outputs. We address this with a two-phase protocol: first derive all non-output facts, then generate output facts. By delaying decisions until evidence stabilizes, the protocol reduces schedule-dependent divergence.

Theoretical background. Let $\Sigma = \Sigma^{\mathsf{evid}} \cup \Sigma^{\mathsf{dec}}$, where

$$\Sigma^{\mathsf{dec}} = \{\varphi = Q[\bar{x}](X \to p_0) \in \Sigma \mid X \in P_{\mathsf{out}}\},$$

and $\Sigma^{\mathsf{evid}} = \Sigma \setminus \Sigma^{\mathsf{dec}}$. Under the two-phase protocol, Meta Chase executes: $\Gamma^{\mathsf{evid}} = \mathsf{Chase}(G, \Sigma^{\mathsf{evid}}, \Gamma_0)$, followed by $\Gamma^* = \mathsf{Chase}(G, \Sigma^{\mathsf{dec}}, \Gamma^{\mathsf{evid}})$. That is, output predicates are derived only after the evidence closure is fully computed. This enforces a partial order on rule applications, decoupling output generation from task-specific scheduling decisions.

Example 9: Revisit Example 4, where two task-specific subsets derive $x.\mathit{risk} = \mathit{thermal}$ and $x.\mathit{recoverable} = \mathit{true}$, followed by decision rules: $x.\mathit{risk} = \mathit{thermal} \rightarrow x.\mathit{action} = \mathit{discard}$, and $x.\mathit{recoverable} = \mathit{true} \rightarrow x.\mathit{action} = \mathit{rework}$.

Under single-phase execution, if the discard rule fires first, $\mathit{locked}(x)$ is derived, and the rework rule becomes disabled, yielding $x.\mathit{action} = \mathit{discard}$. Reversing the order yields $x.\mathit{action} = \mathit{rework}$. Hence outputs depend on schedule.

Under the two-phase protocol, all evidence rules (deriving *risk* and *recoverable*) are executed to closure first, yielding a fixed evidence set Γ_{evid}. Decision rules are then applied over Γ_{evid}. As a result, decision applicability no longer depends on the deduction order. Indeed, if both decision rules are enabled in the second phase, either both action facts are derived and resolved by canonicalization (Section V-B), or a fixed deterministic priority rule is applied to select one action; in either case, the final published output is schedule-independent. □

Complexity. When Σ^{evid} and Σ^{dec} are fixed rule sets, each chase invocation runs in polynomial time in $|G|$ under GARs. Thus the two-phase protocol adds at most one additional chase over Σ^{dec}, yielding overall runtime $O(|G|^c)$, for some constant c depending on the largest pattern size. Hence the protocol incurs no asymptotic overhead in data complexity.

Algorithm. We present Algorithm TwoPhaseChase for the two-phase protocol in Figure 4. Starting from Γ_0, the protocol first computes the evidence closure Γ^{evid} by repeatedly applying GARs in Σ^{evid} until no valid chasing step exists (line 2; *i.e.,* the procedure chase). It then applies GARs in Σ^{dec} over Γ^{evid} to derive decision facts (line 3). This defers all output derivations to the final phase of deduction. The facts deduced via Σ^{dec} may deduce new evidences using Γ^{evid}. Then it repeats these two steps until no further evidence rules are activated afterward, *i.e.,* no fact can be derived (lines 5-9). To accelerate the computation, we adopt the following strategies: (1) to avoid recomputing matches of GARs, we construct the critical fact set $\mathcal{C}_\Sigma$. (2) We adopt incremental computation to avoid redundancy (procedure IncChase, lines 6-7). More specifically, after deducing facts using Γ^{evid} (resp. Σ^{dec}), it triggers the chase only on newly generated facts in Γ^{evid} (resp. Σ^{dec}), rather than invoke chase steps on all facts in Γ^{evid} (resp. Σ^{dec}).

Example 10: Consider the GAR set $\Sigma = \Sigma_1 \cup \Sigma_2$ in Example 6. When $P_{\mathsf{out}} = \{\mathit{action}(\cdot)\}$, the set Σ can be divided into two categories: $\Sigma^{\mathsf{dec}} = \Sigma_2$ and $\Sigma^{\mathsf{evid}} = \Sigma_1$. Given the GAR set Σ, a graph G and a set Γ of validated facts, TwoPhaseChase first applies GARs in Σ^{evid} to deduce fact $v.\mathsf{status} = \mathsf{anomalous}$, from which predicate $v.\mathsf{action} = \mathsf{requiresInspection}$ can be deduced using GARs in Σ^{dec}. □

Input: A graph G and a set of tasks $\mathcal{S} = \{\Sigma_W \mid W \in \mathcal{W}\}$.
Output: The set Γ^* of derived facts.
1. divide the GAR set Σ into Γ^{evid} and Γ^{dec};
2. $\Gamma^{\mathsf{evid}} := \mathsf{chase}(G, \Sigma^{\mathsf{evid}}, \Gamma)$;
3. $\Gamma^{\mathsf{dec}} := \mathsf{chase}(G, \Sigma^{\mathsf{dec}}, \Gamma \cup \Gamma^{\mathsf{evid}})$;
4. $\Gamma := \Gamma^{\mathsf{evid}} \cup \Gamma^{\mathsf{dec}}$; $\Delta\Gamma := \Gamma$;
5. **while** $\Delta\Gamma \neq \emptyset$ **do**
6. $\quad \Delta\Gamma^{\mathsf{evid}} := \mathsf{IncChase}(G, \Sigma^{\mathsf{evid}}, \Gamma)$;
7. $\quad \Delta\Gamma^{\mathsf{dec}} := \mathsf{IncChase}(G, \Sigma^{\mathsf{dec}}, \Gamma \cup \Delta\Gamma^{\mathsf{evid}})$;
8. $\quad \Delta\Gamma := \Delta\Gamma^{\mathsf{evid}} \cup \Delta\Gamma^{\mathsf{dec}}$;
9. $\quad \Gamma := \Gamma \cup \Delta\Gamma$; $\Gamma^{\mathsf{dec}} := \Gamma^{\mathsf{dec}} \cup \Delta\Gamma^{\mathsf{dec}}$;
10. **return** Γ^{dec};

Fig. 4: Algorithm TwoPhaseChase

Complexity. TwoPhaseChase takes $O(|G|^2 f(G, \mathcal{S}))$ time to compute the set Γ^*, where $f(G, \mathcal{S})$ is the complexity to enumerate all matches of graph patterns Q in the graph G for all GAR $Q[\bar{x}](X \rightarrow p_0)$ in $\mathcal{S}$. Observe that (1) dividing Σ into Γ^{evid} and Γ^{dec} takes $O(|\Sigma|)$ time; and (2) computing all derived facts takes $O(|G|^2 f(G, \mathcal{S}))$ time, since (a) there exist at most $O(|G|^2)$ many derived facts, *i.e.,* at most $O(|G|^2)$ chase steps; and (b) each chase step takes $O(f(G, \mathcal{S}))$ time.

Performance guarantees. Let Γ^*_T and $\Gamma^*_{T'}$ be terminal outputs produced under two fair schedules under the protocol. Then

$$\Gamma^*_T \cap P_{\mathsf{out}} = \Gamma^*_{T'} \cap P_{\mathsf{out}}.$$

That is, output-level confluence is ensured by design. Moreover, all derived outputs remain logically sound, as they are obtained from valid GAR applications over the evidence closure.

VI. Experimental Study

Using real-life graphs and workloads, we evaluate the effectiveness and efficiency of AFF for workload-driven deduction.

Experimental setting. We start with the experimental setting.

Datasets. We used three real-world graphs: (a) DBpedia [17], a cross-domain graph extracted from Wikipedia, consisting of 7.96M vertices, 22.8M edges and 633 distinct attributes; (b) YAGO3 [18], a graph derived from Wikipedia, WordNet and GeoNames, with 123K vertices, 1.09M edges and 37 attributes; and (c) IMDB [19], a graph of films, people and companies with 347K vertices, 985K edges and 8 attributes.

To test the scalability of Meta-chase, we generated synthetic graphs $G = (V, E, \mathcal{L}, \ell, \mathcal{A}, \lambda)$ using the LDBC benchmark generator [20], controlled by the number $|V|$ of vertices (up to 10.6M) and the number $|E|$ of edges (up to 41M).

Graph analytic tasks. We evaluated AFF on tasks that stress workload adaptivity, schedule robustness and output consistency. The tasks include entity classification, relationship inference, entity linking, and recommendation-oriented reasoning. Specifically, the tasks infer occupation, category and affiliation for DBpedia, type hierarchy, geographic relation and membership for YAGO3, and collaboration, influence and recommendation links for IMDB. We further evaluated competing decision scenarios, cross-task fact reuse and dynamic workload shifts, as well as the proposed confluence mechanisms.

Each task derives a set P_{out} of output predicates with a GAR set Σ [21], focusing on decision-aware deduction rather than full closure. The set of P_{out}-relevant rules contains 44, 11 and 17 rules for DBpedia, YAGO3 and IMDB, respectively. These rules are partitioned into strongly connected components (SCC)-based tasks in the rule-dependency graph, yielding up to 200 tasks for policy-driven scheduling and fact sharing.

Baselines. We compared AFF with two deduction strategies: (1) FF [2], which performs a chase of the entire GAR set to closure without task decomposition; and (2) FF_{task}, a task-aware baseline that re-runs the entire rule set for each task. All methods were implemented on a common GAR engine with a C++/CUDA graph-matching core and a Python task-orchestration layer. For policy evaluation, we additionally compared the policy-generated schedules of AFF with random fair schedules (random), and report the mean over 5 runs.

Each objective was evaluated under three decision modes: EC (early commit), canon (canonicalization; Section V-B), and two–phase (evidence-first, decide-last; Section V-C). For schedule-robustness tests, we generated five random topological schedules per dataset and report the average results.

Effectiveness metrics. We evaluated both correctness and efficiency. As ground truth, we used the P_{out} facts obtained by chasing the entire rule set to closure. Correctness metrics include: (a) *precision and recall* of the derived P_{out} facts, and (b) *cross-schedule uniqueness*, the fraction of $(\textit{subject}, \textit{predicate})$ keys whose output value is invariant across all fair schedules; a value of 1.0 indicates complete schedule independence.

Efficiency metrics include the following: (c) chasing steps, (d) rule invocations, and (e) wall-clock time. For a fixed rule set, all lossless strategies derive the same closure; hence chasing steps quantify redundant derivation, while rule invocations and runtime reflect scheduling overhead.

Configuration. All experiments were run on a single cluster node with Intel Xeon Gold 6248 @ 2.50GHz and 64 GB of RAM. Each reported result is averaged over three runs.

Experimental results. We next report our findings.

Exp-1: Effectiveness We start with the decision quality.

Output accuracy/utility. We first evaluated the quality of decision outputs, measured by the precision and recall of the derived P_{out} facts against the full closure obtained by chasing the entire rule set. Figure 5(a) reports the results.

(1) The recall of all methods increase as deduction budget (chasing steps) grows, since more applicable GARs are activated and additional P_{out} facts become derivable.

(2) Despite activating only selected GAR subsets, AFF reaches a recall of 0.94 on DBpedia under a budget of 40,000 chasing steps. The policy focuses deduction on reasoning paths that contribute directly to P_{out}, and recovers most relevant facts.

(3) Under the same budget, AFF and FF outperform EC by 37.2% and 1.9%, respectively. The reason is that EC commits decisions before all supporting evidence is available, and may thus miss P_{out} facts. In contrast, AFF and FF derive decisions from more complete evidence, yielding higher output quality.

Output consistency. We next tested schedule robustness, measured by the number of output facts whose values vary across task schedules. As shown in Figure 5(b), (1) inconsistency increases with the number of chasing steps, as longer deduction chains create more opportunities for schedule-dependent interactions; and (2) AFF yields fewer inconsistent outputs than random schedules, as policy-driven scheduling promotes stable task ordering and fact reuse across related deductions.

Robustness to workload shifts. We tested whether AFF remains effective when the workloads change. Different workloads were generated by randomly partitioning the GAR set Σ into 150 tasks. As shown in Figure 5(c), (1) different partitions induce different deduction paths; but (2) AFF consistently recovers similar outputs. On DBpedia, under a budget of 40,000 chasing steps, all partitions recover at least 88% of the P_{out} facts derived by full chase. This is because policy-driven scheduling adapts to the current deduction state, making AFF less sensitive to the particular task decomposition.

Decision stability. We next evaluated the robustness of decision outputs to schedule perturbations. Given a rule collection $\mathcal{S}$, we generated $\mathcal{S}_{p\%}$ by randomly shuffling $p\%$ of its GAR activations. We measured the overlap of the resulting P_{out} facts.

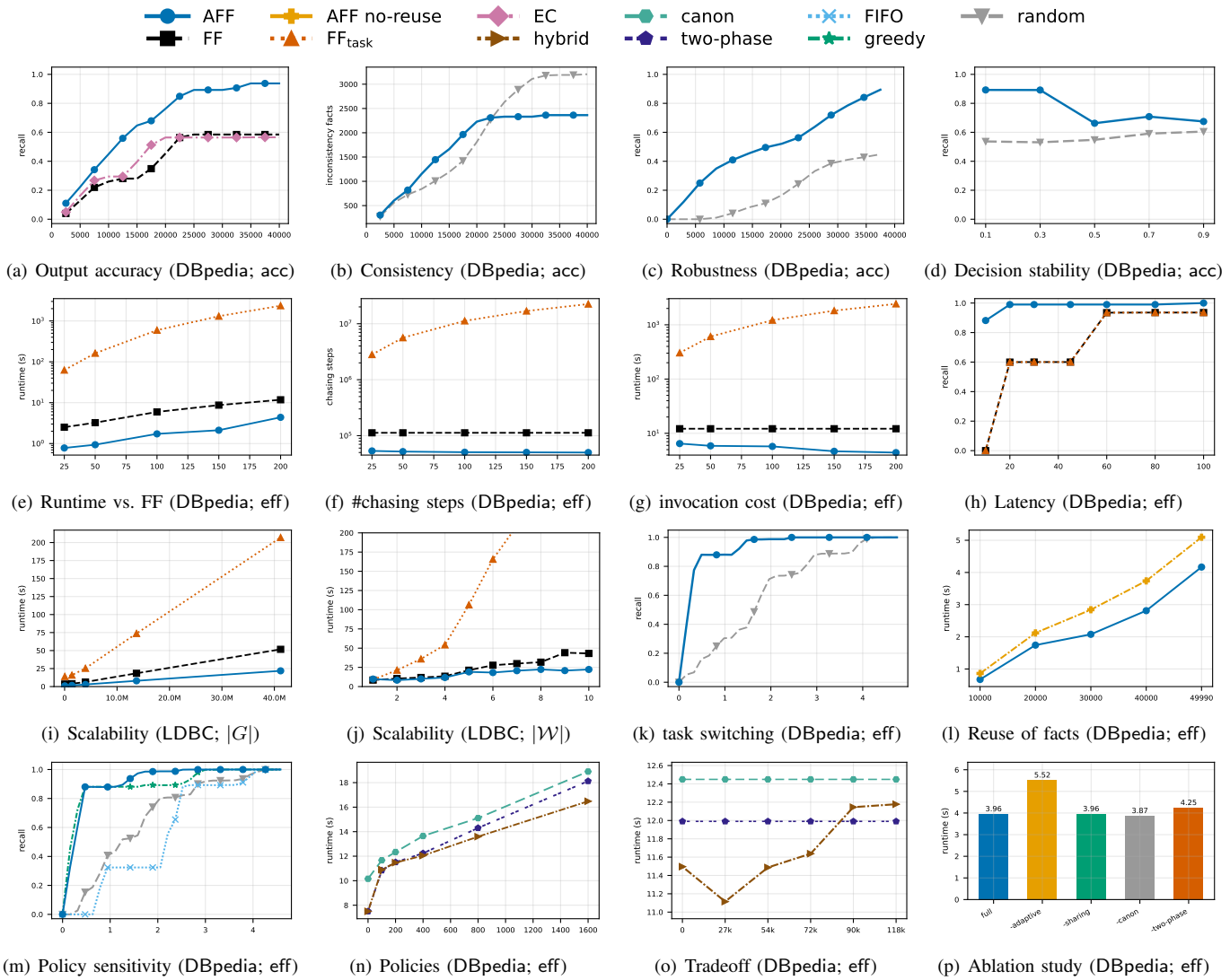
As shown in Figure 5(d), (1) output divergence increases with p, as larger perturbations alter more deduction paths; and (2) AFF is more stable than random by 20.5%. On DBpedia, even for $p = 90\%$, $\mathcal{S}_{p\%}$ recovers 68% of the P_{out} facts derived under $\mathcal{S}$. This is because the policy preserves output-relevant deduction paths and reuses shared facts across tasks, whereas random ignores task dependencies. Consequently, schedule perturbations have a smaller impact on the outputs of AFF.

Exp-2: Efficiency. We next tested execution performance.

Runtime vs. FF. We evaluated scalability by varying the task load from 25 to 200. As shown in Figure 5(e), (1) runtime increases with task load for both methods; and (2) AFF is up to 2.69× faster than FF on DBpedia, since it activates only output-relevant GAR subsets, whereas FF chases the full set.

Number of chasing steps. We next measured deduction effort using the number of chasing steps. As shown in Figure 5(f), (1) AFF consistently outperforms both FF and FF_{task}, as it activates only task-relevant GAR subsets; and (2) chasing steps decrease with the number of tasks, since finer task decompositions reduce output-irrelevant derivations.

Rule invocation cost. We next evaluated execution overhead by measuring runtime under different task decompositions. We varied the number of tasks from 25 to 200. As shown in Figure 5(g), (1) AFF consistently beats both FF and FF_{task}, by *e.g.,* up to 2.8× and 550× on DBpedia, respectively. (2) Its runtime decreases as the number of tasks increases, since AFF activates only output-relevant GAR subsets and reuses derived facts across tasks, whereas FF chases the full rule set and FF_{task} repeatedly re-executes it for individual tasks.



(a) Output accuracy (DBpedia; acc) (b) Consistency (DBpedia; acc) (c) Robustness (DBpedia; acc) (d) Decision stability (DBpedia; acc)

(e) Runtime vs. FF (DBpedia; eff) (f) #chasing steps (DBpedia; eff) (g) invocation cost (DBpedia; eff) (h) Latency (DBpedia; eff)

(i) Scalability (LDBC; $|G|$) (j) Scalability (LDBC; $|\mathcal{W}|$) (k) task switching (DBpedia; eff) (l) Reuse of facts (DBpedia; eff)

(m) Policy sensitivity (DBpedia; eff) (n) Policies (DBpedia; eff) (o) Tradeoff (DBpedia; eff) (p) Ablation study (DBpedia; eff)

Fig. 5: Performance evaluation

Latency under budget constraints. We next tested the utility of AFF under budgets. We varied the runtime budget from 10% to 100% of the full-chase runtime and measured the number of derived P_{out} facts. As shown in Figures 5(h), (1) both the number of chasing steps and derived facts increase with the budget; and (2) under the same budget, AFF derives more P_{out} facts than FF and FF_{task}. This is because AFF prioritizes output-relevant tasks based on the current deduction state, but the baselines spend budget on less relevant deductions.

Scalability w.r.t. graph and workload sizes. We evaluated the scalability of AFF, FF and FF_{task} by varying the synthetic graph size from 36K to 41M and the workload size from 1 to 10. Unless stated otherwise, we fixed $|G|$=13.7M and $|\mathcal{W}|$=4. As shown in Figure 5(i) and 5(j), (1) the runtime of all methods increases with graph and workload sizes, since more facts, rules, and tasks must be processed; and (2) AFF consistently outperforms both FF and FF_{task}, since it activates only task-relevant GAR subsets and reuses derived facts, whereas FF performs full chase and FF_{task} repeatedly re-chases the rule set. The larger the graph or workload, the bigger the gap.

Exp-3: Adaptivity. We evaluated the ability of AFF to adapt task scheduling to the current deduction state and workload context, thus improving deduction efficiency and output utility.

Performance under task switching. We compared AFF's policy with random and fixed task schedules, and report the change of recall with increasing runtime. As shown in Figure 5(k), (1) different task orders yield substantially different runtime and utilities, showing that scheduling decisions have a significant impact on deduction; and (2) AFF consistently outperforms the baselines. This is because its policy exploits the current deduction state to prioritize tasks that are more likely to derive new facts and benefit subsequent deductions, whereas random schedules ignore task dependencies and fixed schedules cannot adapt to evolving deduction states.

Reuse of intermediate facts across tasks. We evaluated the impact of fact sharing across tasks. As shown in Figure 5(l), (1) when tasks are repeatedly invoked across related deductions, AFF is 22% faster than a no-reuse variant on DBpedia; and (2) the benefit increases with the number of chasing

steps. The reason is that shared intermediate facts are derived once and reused across tasks, avoiding repeated invocations.

Policy sensitivity. We compared AFF with three GAR-selection strategies: (a) random, randomly selects a task among all currently eligible tasks; (b) FIFO: selects the earliest eligible task using a fixed topological ordering; and (c) greedy: selects the task with the largest immediate utility, *e.g.,* the estimated number of newly derivable facts. As shown in Figure 5(m), (1) deduction utility varies significantly across policies; and (2) under a budget of 50% of the full runtime, the learned policy performs best, achieving a recall of 0.99 on DBpedia. This is because it adapts task selection to the current deduction state, whereas the baselines use fixed or local heuristics.

Exp-4: Confluence mechanisms. We next evaluated the effectiveness of three algorithms to enforce output-level confluence.

Output uniqueness across schedules. We first applied algorithm CheckCommunity (Section V-A) to determine whether a workload is output-commutative. We then generated multiple schedules and measured output uniqueness. We found the following (not shown): (1) the cross-schedule uniqueness rate was 0.91, 0.94 and 1.00 on DBpedia, YAGO3 and IMDB, respectively; (2) all workloads identified as output-commutative produced identical outputs across schedules; and (3) non-commutative ones often generated distinct outputs, motivating the need for canonicalization and the two-phase protocol.

Overhead of canonicalization and two-phase deduction. We next evaluated the cost of our confluence mechanisms. (1) canon incurs negligible overhead. On DBpedia, it accounts for only 1% of the total runtime (4.24s, not shown), since it requires only sorting and local comparison over the output facts; and (2) two–phase is also efficient. On DBpedia, computing Σ^{dec} and Σ^{evid} takes less than 0.01% of the 3.88s chase. Moreover, AFF without the two-phase decomposition is 8.2% slower (4.20s), since decision and evidence rules are interleaved and trigger additional deductions.

Choosing enforcement policies. We also evaluated when each confluence mechanism should be used. We measured output inconsistency, runtime overhead, and output recall under three policies: canon, two–phase, and a hybrid policy that invokes canon when the inconsistency rate exceeds a threshold after each round of two–phase (see Algorithm TwoPhaseChase). To show the impact of inconsistency, we injected N_c conflicts into the fact sets, and varied N_c from 0 to 1600. As shown in Figure 5(n), two–phase is preferable when conflicts are sparse, since it deduces all facts and is as efficient as hybrid, while canon may miss some facts due to canonicalization. The hybrid policy achieves the best tradeoff when conflicts grow dense; *i.e.,* it is 9% faster than two–phase, since delaying canonicalization prevents schedule-dependent divergence.

Tradeoff between adaptivity and consistency. We evaluated the impact of consistency enforcement on the performance of AFF. We adopted a threshold $\mathcal{I}$ on the number of inconsistent output facts. Whenever the number of inconsistencies exceeds $\mathcal{I}$, Canon is invoked to restore consistency. We varied $\mathcal{I}$ from 0 to $118k$. To show the impact of inconsistency, we injected N_c=1600 conflicts into the fact sets. As shown in Figure 5(o), (1) runtime shows a non-monotonic trend as $\mathcal{I}$ increases. When $\mathcal{I}$ is small, Canon is triggered frequently, introducing overhead. When $\mathcal{I}$ is large, more conflicting outputs are allowed to accumulate, leading to additional schedule-dependent deductions. (2) AFF achieves the best performance at $\mathcal{I} = 27$k, balancing between adaptivity and consistency with a runtime of 11.1s, when we injected N_c=1600 conflicts into the data.

Exp-5: Ablation study. We evaluated the contribution of the major components of AFF by comparing the following variants of AFF: (a) full AFF; (b) AFF without adaptive policy learning; (c) AFF without fact sharing; (d) AFF without canonicalization; and (e) AFF without the two-phase protocol.

As shown in Figure 5(p), (1) full AFF performs best; (2) disabling adaptive policy learning causes the largest slowdown (39%), since task selection becomes less effective at prioritizing output-relevant deductions; (3) removing the two-phase protocol slows execution by 7%, since decision rules may be repeatedly activated before the required evidence has been derived; (4) disabling fact sharing has a negligible effect within a well-ordered pass; its benefit becomes evident when tasks are repeatedly invoked (Exp-3.2); and (5) removing canonicalization yields only a marginal ($\approx 2\%$) speedup, since canonicalization is a lightweight post-processing phase whose cost is negligible compared with the chase.

Summary. The experiments verify the effectiveness of key components of AFF. (1) AFF achieves high output quality: its recall reaches 0.94 under a $40{,}000$-step budget on DBpedia. (2) By combining policy-driven GAR selection and cross-task fact sharing, AFF is up to $2.69\times$ faster than FF. (3) Adaptive scheduling consistently outperforms static and random schedules, demonstrating the value of state-aware task selection. (4) Our confluence mechanisms incur negligible overhead while ensuring output consistency. (5) AFF scales to graphs with 10.6M vertices and 41M edges and workloads of 200 tasks.

VII. Related Work

We review related work from the perspective of *logical reasoning under dynamic rule sets*. Prior work typically assumes a fixed rule set, under which inference semantics such as closure and confluence are well-defined. In contrast, this work studies deduction where rule subsets are selected dynamically by policy and workload, which changes the reasoning semantics.

Rule-based deduction and chase frameworks. Chase and its variants are widely used for rule-based deduction in databases. Prior work has studied the standard chase [4], [22], oblivious chase [23], semi-oblivious chase [24], core chase [25], and unifying frameworks such as X-chase [26], as well as termination and boundedness questions for tuple generating dependencies and related rule classes [27], [28], [29], [30], [31], [32], [33], [34], [35], [36]. The previous approaches assume a fixed rule set, under which the closure is defined independently of policy.

Meta-Chase departs from this setting by executing dynamically selected rule subsets. As a result, the resulting closure is no longer uniquely determined by the input graph and rule set, but may depend on the execution schedule and policy. This invalidates the Church-Rosser guarantee underlying fixed-rule chase, and introduces new challenges for termination and output-level confluence that are not addressed in prior work. To this end, we develop a formal framework for Meta-Chase, establish termination guarantees under fair schedules, and propose mechanisms to ensure output-level consistency, supported by complexity analysis and efficient algorithms.

Multi-agent and policy-driven reasoning systems. Previous work on the subject studies policy specification for coordinating agent behavior, controlling cooperation, and constraining agent actions [37], [38], [39], [40]. Recent work further improves agent performance via cognitive models and LLM-based components, or evaluates agent frameworks through simulators and industrial criteria [41], [42], [43], [44].

In contrast, AFF does not treat policy just as a coordination layer over black-box agents. Instead, policy determines which rule subsets are activated during deduction, and thus directly affects the semantics of inference. We focus on not only agent orchestration, but also the formal properties of policy-driven deduction, including termination and output-level confluence.

Adaptive workflow and scheduling in logical inference. There is also substantial work on workflow management and scheduling for improving inference efficiency and resource utilization, including policy-based workflow control [45], [46], [47], priority-aware scheduling [48], GPU/CPU inference scheduling [49], [50], [51], task-graph decomposition [52], and resource-aware optimization [53], [54]. These approaches optimize latency, throughput, or memory usage under the assumption that scheduling does not change the logical outcome.

AFF differs in that scheduling determines which rule subsets are activated and in what order; as a result, it may directly affect the resulting fact set. This introduces a new dimension in which scheduling impacts not only efficiency but also correctness, thus requiring explicit conditions for consistency.

Neuro-symbolic and decision-aware reasoning. Neuro-symbolic systems combine learning and reasoning by embedding logical structures into neural models, *e.g.,* via neural-symbolic representations, rule layers, user feedback, and differentiable modules [55], [56], [57], [58], [59], [60]. These approaches primarily focus on incorporating logical constraints into the *training or inference of ML models*, aiming to improve representation quality, interpretability, or robustness.

In contrast, GARs follow the dual paradigm: rather than embedding logic into learning, we embed ML predicates into a logical reasoning framework. GARs treat ML models as predicates within rule-based deduction, so that inference is controled by logical semantics while leveraging data-driven signals. This makes ML an integral part of deductive reasoning, rather than a learning component. Hence, the challenge is not representation learning, but understanding how dynamic rule selection affects termination and confluence, and how to restore output-level consistency under schedule-dependent execution, which are not addressed in neuro-symbolic systems.

Dynamic and order-sensitive reasoning. Related are studies on active rules, rewriting systems and incremental reasoning.

Active rules and conflict handling. Production rules and event-condition-action (ECA) systems allow dynamically triggered rules and may exhibit order-dependent behavior due to conflicts [61], [62], [63], [64], [65]. These systems typically rely on heuristic conflict resolution (*e.g.,* priorities [62], meta-rule [64], [65] or semantic analysis [63]), without formal guarantees on termination or confluence. In contrast, we provide a formal treatment of rule interaction under dynamic selection, with guarantees on termination and output-level consistency.

Rewriting systems and confluence. Confluence and Church-Rosser properties have been well studied in terms of rewriting systems under fixed rewrite rules [66], [67], [68], [69]. In contrast, Meta-Chase considers dynamically selected rule subsets, which invalidate classical confluence assumptions and require new mechanisms to restore output-level consistency.

Incremental and dynamic reasoning. Prior work on incremental reasoning focuses on maintaining inference results under data updates with a fixed rule set [70], [71], [72], incremental algorithms [73], [74], [75] and ML models [76], [77], [78], [79]. In contrast, we study the dual setting where rules evolve while data remains fixed, fundamentally affecting the semantics of deduction rather than only its efficiency.

To the best of our knowledge, none of these lines of work addresses deduction under dynamically evolving rule subsets, where policy-driven rule selection may affect inference outcomes. This gap motivates this work on Meta Chase.

VIII. Conclusion

We have introduced Agentized Fishing Fort (AFF), a policy-driven extension of Fishing Fort (FF) for adaptive GAR-based deduction under dynamic workloads. We have formalized Meta Chase as a foundation for policy-driven reasoning over dynamically selected rule subsets, and established its termination under all fair schedules. To address the loss of global confluence, we have developed complementary mechanisms based on output commutativity, canonicalization, and two-phase deduction that guarantee output-level consistency despite schedule-dependent execution. These results move graph deduction beyond static closure computation toward adaptive, policy-driven reasoning with formal guarantees. The effectiveness is validated by experimental study on real-life graphs.

Future work includes extending AFF to support utility-aware scheduling under resource constraints, where policies dynamically select GAR subsets to maximize utility while preserving termination and output-level consistency. Another topic is to learn schedule-robust execution motifs that produce stable outputs across workload shifts and policy changes. Such motifs could improve the reliability of adaptive deduction and enable learning-guided optimization with formal guarantees.

IX. AI-Generated Content Acknowledgment

The content (including but not limited to text, figures, images, and code) was not generated by artificial intelligence.

References

[1] Shenzhen Institute of Computing Sciences, "Fishing fort," 2022, *https://en.sics.ac.cn/col84/index*.

[2] W. Fan and S. Liu, "Fishing fort: A system for graph analytics with ML prediction and logic deduction," in *The Provenance of Elegance in Computation - Essays Dedicated to Val Tannen, Tannen's Festschrift*, ser. OASIcs, vol. 119. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, pp. 6:1–6:18.

[3] W. Fan, R. Jin, M. Liu, P. Lu, C. Tian, and J. Zhou, "Capturing associations in graphs," *PVLDB*, vol. 13, no. 11, pp. 1863–1876, 2020.

[4] F. Sadri and J. D. Ullman, "The interaction between functional dependencies and template dependencies," in *SIGMOD*, 1980.

[5] W. Fan, D. Li, P. Liang, S. Liu, Y. Wang, Y. Wang, M. Xie, and R. Zhang, "Graph association analyses for early drug discovery," *PVLDB*, vol. 17, no. 12, pp. 4293–4296, 2024.

[6] W. Fan, Y. Leng, D. Li, S. Liu, M. Ouyang, Y. Wang, M. Xie, and Q. Yuan., "DreamCreek: AI for battery formation and grading," in *ICDE*. IEEE, 2025.

[7] L. Fan, W. Fan, P. Lu, C. Tian, and Q. Yin, "Enriching recommendation models with logic conditions," *Proc. ACM Manag. Data*, 2024.

[8] W. Fan, L. Fan, D. Lin, and M. Xie, "Explaining GNN-based recommendations in logic," *PVLDB*, vol. 18, no. 3, pp. 715–728, 2025.

[9] K. Pang, W. Fan, M. Xie, and D. Lin, "Explaining GNN negatives globally and locally," in *ICDE*, 2026.

[10] S. Abiteboul, R. Hull, and V. Vianu, *Foundations of Databases*. Addison-Wesley, 1995.

[11] "Full version," 2026, https://anonymous.4open.science/r/meta-chase-CA89/.

[12] M. Garey and D. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, 1979.

[13] C. H. Papadimitriou, *Computational Complexity*. Addison-Wesley, 1994.

[14] H. Spakowski, "Completeness for parallel access to NP and counting class separations," Ph.D. dissertation, University of Düsseldorf, Germany, 2005.

[15] W. Fan, P. Lu, C. Tian, and J. Zhou, "Deducing certain fixes to graphs," *PVLDB*, vol. 12, no. 7, pp. 752–765, 2019.

[16] E. Fredkin, "Trie memory," *Commun. ACM*, vol. 3, no. 9, pp. 490–499, 1960.

[17] J. Lehmann, R. Isele, M. Jakob, A. Jentzsch, D. Kontokostas, P. N. Mendes, S. Hellmann, M. Morsey, P. van Kleef, S. Auer, and C. Bizer, "DBpedia - A large-scale, multilingual knowledge base extracted from Wikipedia," *Semantic Web*, vol. 6, no. 2, pp. 167–195, 2015.

[18] F. Mahdisoltani, J. Biega, and F. M. Suchanek, "YAGO3: A knowledge base from multilingual Wikipedias," in *CIDR*, 2015.

[19] "Import IMDB," *https://github.com/bitnine-oss/import_imdb*.

[20] O. Erling, A. Averbuch, J. Larriba-Pey, H. Chafi, A. Gubichev, A. Prat-Pérez, M. Pham, and P. A. Boncz, "The LDBC social network benchmark: Interactive workload," in *SIGMOD*, 2015, pp. 619–630.

[21] W. Fan, X. Wang, Y. Wu, and J. Xu, "Association rules with graph patterns," *PVLDB*, vol. 8, no. 12, pp. 1502–1513, 2015.

[22] D. Maier, A. O. Mendelzon, and Y. Sagiv, "Testing implications of data dependencies," *TODS*, vol. 4, no. 4, pp. 455–469, 1979.

[23] A. Calì, G. Gottlob, and M. Kifer, "Taming the infinite chase: Query answering under expressive relational constraints," *J. Artif. Intell. Res.*, vol. 48, pp. 115–174, 2013.

[24] B. Marnette, "Generalized schema-mappings: From termination to tractability," in *PODS*, 2009, pp. 13–22.

[25] A. Deutsch, A. Nash, and J. B. Remmel, "The chase revisited," in *PODS*, 2008, pp. 149–158.

[26] S. Delivorias, "Chase variants & boundedness," Ph.D. dissertation, Université Montpellier, 2019.

[27] S. Greco and F. Spezzano, "Chase termination: A constraints rewriting approach," *PVLDB*, vol. 3, no. 1-2, pp. 93–104, 2010.

[28] T. Gogacz and J. Marcinkowski, "All–instances termination of chase is undecidable," in *ICALP*, 2014, pp. 293–304.

[29] M. Calautti, G. Gottlob, and A. Pieris, "Chase termination for guarded existential rules," in *PODS*, 2015, pp. 91–103.

[30] M. Calautti, G. Gottlob, and A. Pieris, "Non-uniformly terminating chase: Size and complexity," in *PODS*, 2022, pp. 369–378.

[31] M. Calautti and A. Pieris, "Semi-oblivious chase termination: The sticky case," *Theory of Computing Systems*, vol. 65, no. 1, pp. 84–121, 2021.

[32] T. Gogacz, J. Marcinkowski, and A. Pieris, "All-instances restricted chase termination," in *PODS*, 2020, pp. 245–258.

[33] D. Carral, I. Dragoste, M. Krötzsch, and C. Lewe, "Chasing sets: How to use existential rules for expressive reasoning," in *Description Logics*, 2019.

[34] D. Carral, I. Dragoste, and M. Krötzsch, "Restricted chase (non)termination for existential rules with disjunctions," in *IJCAI*, 2017, pp. 922–928.

[35] A. Alizad, M. Calautti, M. Milani, and A. Pieris, "Semi-oblivious chase termination for linear existential rules and beyond: An experimental analysis," *TODS*, 2025.

[36] D. Carral, L. Gerlach, L. Larroque, and M. Thomazo, "Restricted chase termination: You want more than fairness," *Proc. ACM Manag. Data*, vol. 3, no. 2, pp. 109:1–109:17, 2025.

[37] Y.-B. Peng, J. Gao, J. Hu, and B.-S. Liao, "Policy-driven agent social," in *International Conference on Machine Learning and Cybernetics*, vol. 1, 2005, pp. 345–350.

[38] J. Umugwaneza and J. Hu, "Multi-agent oriented policy-based management mechanism," in *International Conference on Computer Engineering and Technology*, vol. 7, 2010, pp. V7–501.

[39] A. Toniolo, T. J. Norman, and K. Sycara, "Argumentation schemes for policy-driven planning," in *TAFA*, 2011.

[40] R. Ashri, M. Luck, and M. d'Inverno, "On identifying and managing relationships in multi-agent systems," in *IJCAI*, 2003, pp. 743–748.

[41] Z. Shi, J. Zhang, J. Yue, and X. Yang, "A cognitive model for multi-agent collaboration," *Int'l J. Intelligence Science*, vol. 4, pp. 1–6, 2014.

[42] X. He, D. Wu, Y. Zhai, and K. Sun, "Sentinelagent: Graph-based anomaly detection in multi-agent systems," *arXiv preprint arXiv:2505.24201*, 2025.

[43] A. Morris, P. Giorgini, and S. Abdel-Naby, "Simulating BDI-based wireless sensor networks," in *IAT*, 2009, pp. 78–81.

[44] T. S. López, A. Brintrup, D. McFarlane, and D. Dwyer, "Selecting a multi-agent system development tool for industrial applications: A case study of self-serving aircraft assets," in *DEST*, 2010, pp. 400–405.

[45] A. Corradi, N. Dulay, R. Montanari, and C. Stefanelli, "Policy-driven management of agent systems," in *POLICY*, vol. 1995, 2001, pp. 214–229.

[46] P. Senkul, M. Kifer, and I. H. Toroslu, "A logical framework for scheduling workflows under resource allocation constraints," in *VLDB*, 2002, pp. 694–705.

[47] Y. Zeng, R. Zhou, L. Jiao, and R. Zhang, "Online scheduling of edge multiple- model inference with DAG structure and retraining," in *INFOCOM*, 2025, pp. 1–10.

[48] J. Huang, Y. Xiong, X. Yu, W. Huang, E. Li, L. Zeng, and X. Chen, "Slo-aware scheduling for large language model inferences," *CoRR*, vol. abs/2504.14966, 2025.

[49] Z. Han, R. Zhou, C. Xu, Y. Zeng, and R. Zhang, "*InSS*: An intelligent scheduling orchestrator for multi-GPU inference with spatio-temporal sharing," *IEEE Trans. Parallel Distributed Syst.*, vol. 35, no. 10, pp. 1735–1748, 2024.

[50] Y.-D. Lin, Y.-T. Ling, Y.-C. Lai, and D. Sudyana, "Reinforcement learning for AI as a service: CPU-GPU task scheduling for preprocessing, training, and inference tasks," *IEEE Transactions on Network and Service Management*, 2025.

[51] Z. Li, Y. Chen, and T. Gallagher, "Adaptive workflow scheduling in heterogeneous GPU clusters via deep reinforcement learning," *Multidisciplinary Research in Computing Information Systems*, vol. 5, no. 10, pp. 889–903, 2025.

[52] J. Yu, Y. Ding, and H. Sato, "Dyntaskmas: A dynamic task graph-driven framework for asynchronous and parallel llm-based multi-agent systems," in *International Conference on Automated Planning and Scheduling*, vol. 35, no. 1, 2025, pp. 288–296.

[53] X. Li, Y. Li, Y. Li, T. Cao, and Y. Liu, "FlexNN: Efficient and adaptive DNN inference on memory-constrained edge devices," in *MobiCom*, 2024, pp. 709–723.

[54] P. Jiang, H. Wang, Z. Cai, L. Gao, W. Zhang, R. Ma, and X. Zhou, "Slob: Suboptimal load balancing scheduling in local heterogeneous gpu clusters for large language model inference," *IEEE Transactions on Computational Social Systems*, vol. 11, no. 6, pp. 7941–7951, 2024.

[55] J. S. Dhatterwal, S. Kumar, and S. Choudhary, "Neuro-symbolic computing architectures for trustworthy AGI systems," in *ICMNWC*, 2025, pp. 1–5.

[56] V. Dhanraj and C. Eliasmith, "Improving rule-based reasoning in LLMs using neurosymbolic representations," in *EMNLP*, 2025, pp. 30 589–30 608.

[57] A. Wexford and M. Arendt, "Logical inference under explicit constraint boundaries in hybrid neuro-symbolic systems," *Turquoise International Journal of Educational Research and Social Studies*, vol. 6, no. 1, pp. 6–10, 2025.

[58] W. Stammer, P. Schramowski, and K. Kersting, "Right for the right concept: Revising neuro-symbolic concepts by interacting with their explanations," in *CVPR*, 2021, pp. 3619–3629.

[59] N. Cingillioglu and A. Russo, "pix2rule: End-to-end neuro-symbolic rule learning," in *NeSy*, vol. 2986, 2021, pp. 15–56.

[60] S. Amizadeh, H. Palangi, A. Polozov, Y. Huang, and K. Koishida, "Neuro-symbolic visual reasoning: Disentangling "visual" from "reasoning"," in *ICML*, vol. 119, 2020, pp. 279–290.

[61] G. Feuillade, A. Herzig, and C. Rantsoudis, "Database repair via event-condition-action rules in dynamic logic," in *FoIKS*, ser. Lecture Notes in Computer Science, 2022, pp. 75–92.

[62] M. Calautti, L. Caroprese, S. Greco, C. Molinaro, I. Trubitsyna, and E. Zumpano, "Consistent query answering with prioritized active integrity constraints," in *IDEAS*, 2020, pp. 3:1–3:10.

[63] X. Zhao, C. Wang, P. Cui, and G. Sun, "Operational rule extraction and construction based on task scenario analysis," *Inf.*, vol. 13, no. 3, p. 144, 2022.

[64] L. Caroprese and M. Truszczynski, "Active integrity constraints and revision programming," *Theory Pract. Log. Program.*, vol. 11, no. 6, pp. 905–952, 2011.

[65] S. H. Toman and Z. H. Toman, "Formal service conflict detection through abstraction and refinement," *Clust. Comput.*, vol. 28, no. 7, p. 445, 2025.

[66] J. Ketema and J. G. Simonsen, "Least upper bounds on the size of confluence and church-rosser diagrams in term rewriting and λ-calculus," *ACM Trans. Comput. Log.*, vol. 14, no. 4, pp. 31:1–31:28, 2013.

[67] A. Beckmann and G. Moser, "On complexity of confluence and church-rosser proofs," in *MFCS*, 2024, pp. 21:1–21:16.

[68] C. Chenavier, B. Dupont, and P. Malbos, "Confluence of algebraic rewriting systems," *Math. Struct. Comput. Sci.*, vol. 32, no. 7, pp. 870–897, 2022.

[69] A. Claesson, "From Hertzsprung's problem to pattern-rewriting systems," *Algebraic Combinatorics*, vol. 5, no. 6, pp. 1257–1277, 2022.

[70] W. Fan, C. Tian, Y. Wang, and Q. Yin, "Parallel discrepancy detection and incremental detection," *PVLDB*, vol. 14, no. 8, pp. 1351–1364, 2021.

[71] J. Chen, Y. Sun, S. Song, H. Zhang, and X. Yuan, "Minimum change$\neq$ best cleaning: Parallel and incremental error detection under integrity constraints," *Proc. ACM Manag. Data*, vol. 3, no. 4, pp. 256:1–256:26, 2025.

[72] M. Yan, W. Fan, Y. Wang, and M. Xie, "Enriching relations with additional attributes for ER," *PVLDB*, vol. 17, no. 11, pp. 3109–3123, 2024.

[73] W. Fan, C. Hu, and C. Tian, "Incremental graph computations: Doable and undoable," in *SIGMOD*. ACM, 2017, pp. 155–169.

[74] W. Fan, C. Tian, R. Xu, Q. Yin, W. Yu, and J. Zhou, "Incrementalizing graph algorithms," in *SIGMOD*. ACM, 2021, pp. 459–471.

[75] W. Fan, R. Jin, P. Lu, K. Pang, and W. Yu, "Linking entities across relations and graphs," *TODS*, vol. 49, no. 1, pp. 2:1–2:50, 2024.

[76] D. Zhou, F. Wang, H. Ye, L. Ma, S. Pu, and D. Zhan, "Forward compatible few-shot class-incremental learning," in *CVPR*, 2022, pp. 9036–9046.

[77] Y. Li, P. Moghadam, C. Peng, N. Ye, and P. Koniusz, "Inductive graph few-shot class incremental learning," in *WSDM*, 2025, pp. 466–474.

[78] X. Huang, T. Xie, S. Luo, J. Wu, R. Luo, and Q. Zhou, "Incremental learning with multi-fidelity information fusion for digital twin-driven bearing fault diagnosis," *Eng. Appl. Artif. Intell.*, vol. 133, p. 108212, 2024.

[79] Z. Liu, X. He, B. Huang, and D. Zhou, "Incremental learning-enabled fault diagnosis of dynamic systems: A comprehensive review," *IEEE Trans. Cybern.*, vol. 55, no. 12, pp. 5633–5649, 2025.

APPENDIX

PROOF OF THEOREM 1

We prove the theorem in two steps.

(1) We first show that any chasing sequence terminates. It suffices to show that given a property graph G, a GAR set Σ and a fact set Γ, (a) when applying GARs in Σ on G starting from Γ, only finite many facts can be added to Γ; and (b) chase steps are monotone, *i.e.,* each valid step adds a new fact and never removes an existing fact. If these hold, then $\mathsf{Chase}(G, \Sigma, \Gamma)$ is finite. To prove these statements, let $\mathsf{Fact}(G, \Sigma, \Gamma)$ denote the universe of all ground facts that can possibly appear during deduction on G under Σ given Γ.

(a) We first show that $\mathsf{Fact}(G, \Sigma, \Gamma)$ is finite. From the definition of chasing steps, every derived fact is an instantiation of the consequence p_0 of a GAR $Q[\bar{x}](X \to p_0)$ in Σ under a match of its pattern in G. Since G is finite and the chasing steps only add edges to G, the number of pattern matches is finite. Because each pattern match derives only one fact, only finite facts can be added to Γ. That is, $\mathsf{Fact}(G, \Sigma, \Gamma)$ is finite.

(b) We next show that chase steps are monotone. Indeed, each valid chasing step strictly increases the current fact set by adding a fact not already present.

So, a chasing sequence terminate within $|\mathsf{Fact}(G, \Sigma)|$ steps.

(2) We next show that Meta chase terminates for any fair schedule and any policy π. Given any task sequence $W_1, \ldots, W_k \in \mathcal{W}$, any policy function π and an initial fact set Γ_1, let $T = \langle \Sigma_{W_1}, \Sigma_{W_2}, \ldots, \Sigma_{W_k} \rangle$ be a fair schedule determined by π. We show that Meta-chase terminates by induction on the length of the task schedule T. Indeed, (a) $\mathsf{Chase}(G, \Sigma_{W_i}, \Gamma_i)$ $(i \in [1, k])$ is finite for any GAR set Σ_{W_i} in T; and (b) fairness ensures that any rule subset whose rules remain continuously applicable will eventually be selected.

PROOF OF THEOREM 2

Let $I = (G, \Sigma, \Gamma_0, \mathcal{W}, \pi)$ be an instance of Meta-Chase, and let $\mathcal{P}_{\mathsf{out}}$ be a designated set of *output predicates*. For a (fair) task schedule T over $\mathcal{W}$, let Γ^*_T denote the terminal fact set produced by executing Meta-Chase on I under T. We say that $I \in \mathsf{UniqueClosure}_{\mathsf{out}}(\mathsf{MC})$ iff for all fair schedules T and T' over $\mathcal{W}$, $\Gamma^*_T \cap \mathcal{P}_{\mathsf{out}} = \Gamma^*_{T'} \cap \mathcal{P}_{\mathsf{out}}$.

We show the coNP-hardness by reduction from UNSAT, which is coNP-complete (cf. [12]) and remains coNP-complete when the 3CNF formula Φ is not tautological; indeed, it is in PTIME to check whether Φ is tautological. Given a 3CNF formula Φ, we construct an instance $I_\Phi = (G_\Phi, \Sigma_\Phi, \Gamma_0, \mathcal{W}, \pi)$ such that Φ is unsatisfiable iff $I_\Phi \in \mathsf{UniqueClosure}_{\mathsf{out}}(\mathsf{MC})$, *i.e.,* all task sequences yield the same result. Intuitively, graph G_Φ encodes the structure of Φ, task $\mathcal{W}$ defines all possible truth assignments for variables in Φ, policy π ensures that each variable in Φ is assigned only once, and Σ_Φ deduces a designated predicate $x.\mathsf{setAll} = \mathsf{false}$ only when Φ is satisfiable. Below we give the reduction.

(1) Graph G_Φ and initial facts. The graph G_Φ contains a vertex v_i for each variable x_i, a vertex c_j for each clause C_j and a vertex O to represent whether Φ is satisfiable; for each occurrence of a literal in a clause, we add a labeled edge: (v_i, pos, c_j) if x_i appears in C_j and (v_i, neg, c_j) if $\neg x_i$ appears in C_j; and for each clause C_j, we add a labeled edge $(c_i, \mathsf{clause}, O)$. The initial fact set Γ_0 encodes only this structure (variable/clause identities and literal edges).

(2) Tasks and policy. For each variable x_i, define two task types W_i^T and W_i^F. Intuitively, executing W_i^T (resp. W_i^F) commits the assignment $x_i := \mathsf{true}$ (resp. $x_i := \mathsf{false}$). Let $\mathcal{W} = \{W_i^T, W_i^F \mid i \in [n]\}$. The policy π enables a variable to be committed at most once by using a monotone lock fact: if $(v_i.\mathsf{locked} = \mathsf{true}) \in \Gamma$, then $\pi(W_i^T, \Gamma) = \pi(W_i^F, \Gamma) = \emptyset$; otherwise $\pi(W_i^T, \Gamma) = \Sigma_i^T$ and $\pi(W_i^F, \Gamma) = \Sigma_i^F$. We can verify that for every i, exactly one of W_i^T or W_i^F is eventually executed before $v_i.\mathsf{locked} = \mathsf{true}$ is committed.

(3) GARs. We use the following GARs, defined over a graph pattern that matches variable-clause relationships in G_Φ.

- The commitment rules $v_i.\mathsf{chooseT} = \mathsf{true} \to v_i.\mathsf{locked} = \mathsf{true}$ and $v_i.\mathsf{chooseF} = \mathsf{true} \to v_i.\mathsf{locked} = \mathsf{true}$ use a unary pattern consisting of a single variable node v_i.
- One clause-dissatisfaction rule represent the unique unsatisfied truth-assignment for each clause C_j. For example, if $C_j = x_1 \vee \neg x_2 \vee \neg x_3$, we construct the following rule: $v_1.\mathsf{chooseF} = \mathsf{true} \wedge v_2.\mathsf{chooseT} = \mathsf{true} \wedge v_3.\mathsf{chooseT} = \mathsf{true} \wedge \mathsf{pos}(v_1, c_j) \wedge \mathsf{neg}(v_2, c_j) \wedge \mathsf{neg}(v_3, c_j) \wedge \mathsf{clause}(c_j, O) \to O.\mathsf{satAll} = \mathsf{false}$ use a five-node pattern, indicating that clause C_j is not satisfied by literals x_1, $\neg x_2$ and $\neg x_3$.

By the construction of π and the lock rules, T commits each variable exactly once, yielding a Boolean assignment α_T to Φ. By the clause satisfaction rules, a clause vertex c_j derives $O.\mathsf{satAll} = \mathsf{false}$ under T iff α_S does not satisfy clause C_j.

Correctness of the reduction. We next verify that Φ is unsatisfiable iff $I_\Phi \in \mathsf{UniqueClosure}_{\mathsf{out}}(\mathsf{MC})$.

Only if. If Φ is unsatisfiable, then for every fair schedule S, $O.\mathsf{satAll} = \mathsf{false}$ is always derived, and therefore all terminal closures agree on $\mathcal{P}_{\mathsf{out}}$ (in particular, $O.\mathsf{satAll} \notin \Gamma^*_S$ for all S). Therefore, $I_\Phi \in \mathsf{UniqueClosure}_{\mathsf{out}}(\mathsf{MC})$.

If. If Φ is satisfiable, let T_1 be a fair schedule that commits variables according to a satisfying assignment, and let T_2 be a fair schedule that commits variables according to any non-satisfying assignment. Since Φ is not tautological, the schedule T_2 exists. Then $o.\mathsf{satAll} = \mathsf{false} \notin \Gamma^*_{T_1}$ but $o.\mathsf{satAll} = \mathsf{false} \in \Gamma^*_{T_2}$, so the terminal closures differ on $\mathcal{P}_{\mathsf{out}}$ (*i.e.,* $\Gamma^*_{T_1} \cap \mathcal{P}_{\mathsf{out}} \neq \Gamma^*_{T_2} \cap \mathcal{P}_{\mathsf{out}}$) and $I_\Phi \notin \mathsf{UniqueClosure}_{\mathsf{out}}(\mathsf{MC})$.

Therefore, $\Phi \in$ UNSAT iff $I_\Phi \in \mathsf{UniqueClosure}_{\mathsf{out}}(\mathsf{MC})$, which proves that $\mathsf{UniqueClosure}_{\mathsf{out}}(\mathsf{MC})$ is coNP-hard.

PROOF OF THEOREM 4

We show that deciding whether $\mathcal{S}$ is output-commutative is Π^2_p-complete in combined complexity and $\mathsf{P}^{\mathsf{NP}}_{||}$-complete data complexity (when the validated fact sets are fixed).

Combined complexity. We start with combined complexity.

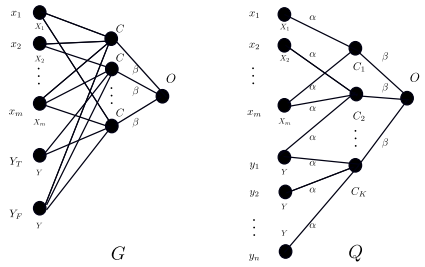


Fig. 6: Graph and patterns in the reduction

Upper bound. Given G and $\mathcal{S}$, we develop the following algorithm to check the complement, *i.e.,* whether $\mathcal{S}$ is *not* output-commutative: (1) Guess a set Γ of validated facts, two GARs Σ_1 and Σ_2 in $\mathcal{S}$, a predicate p_0 and a chase sequence on G that first applies GARs in Σ_1 and then the ones in Σ_2; (2) check whether p_0 is contained in the results of the guessed chasing sequence; if so, continue; otherwise reject the current guess; (3) check whether p_0 cannot be deduced from G via a chasing sequence that first applies GARs in Σ_2 and then GARs in Σ_1; if so, return true. The correctness follows from the definition of the output-communitivity. For the complexity, observe that (a) step (2) is in PTIME when the chase sequence is given; and (b) step (3) is in coNP, since its complement can be done in NP by first guessing a chasing sequence, and then checking whether p_0 can be deduced. Thus, the algorithm is in Σ_p^2, and checking whether $\mathcal{S}$ is output-commutative is in Π_p^2.

Lower bound. We show that checking whether $\mathcal{S}$ is output-commutative is Π_2^p-hard. We reduce from $\mathsf{QBF}_{\forall\exists}$, which is to decide whether $\forall\bar{x}\exists\bar{y}\ \Psi(\bar{x},\bar{y}) = \mathsf{true}$, where $\Psi(\bar{x},\bar{y})$ is a conjunction of clauses, each clause consists of three literals, and each literal is either z or $\neg z$ for a variable z in $\bar{x}$ or $\bar{y}$. The problem is Π_2^p-complete (cf. [13]).

Given $\forall\bar{x}\exists\bar{y}\ \Psi(\bar{x},\bar{y})$, we construct a graph G and a set $\mathcal{S}$ of tasks such that output-commutativity holds for all Γ iff $\forall\bar{x}\exists\bar{y}\ \Psi(\bar{x},\bar{y}) = \mathsf{true}$. Intuitively, we encode an assignment to $\bar{x}$ as a fact set $\Gamma_{\bar{x}}$ (*e.g.,* facts $x_i.\mathsf{value} = \mathsf{true}$ or $x_i.\mathsf{value} = \mathsf{false}$). Construct two rule subsets Σ_1 and Σ_2 such that, for each $\Gamma_{\bar{x}}$, the two chase orders differ on an output fact $f^\star \in P_{\mathsf{out}}$ iff there exists an assignment to $\bar{x}$ satisfying $\forall\bar{y}\Psi(\bar{x},\bar{y}) = \mathsf{false}$. This is achieved by using pattern-matching gadgets to witness the existential choice of $\bar{y}$ and a GAR to make $f^\star$ derivable in exactly one order when the witness exists.

(1) Graph G_Φ. The graph G_Φ is shown in Figure 6. It contains (a) one vertex v_i for each variable x_i, (b) two vertices Y_T and Y_F for the truth assignments for variable y_j, (c) seven vertices c_k ($k \in [1,7]$) labeled C for the seven satisfying truth-assignments for each clause, and (d) a vertex O to represent whether Φ is satisfiable. Vertices $v_1,\ldots,v_m$, Y_T and Y_F are connected to each vertex c_k via edges labeled α, and each vertex c_k is connected to vertex O via an edge labeled β.

(2) GARs. We use the following two sets Σ_1 and Σ_2 of GARs:

- The set Σ_1 consists of two GARs φ_1^1 and φ_2^1, where (I) φ_1^1 encodes the topological structure of $\Psi(\bar{x},\bar{y})$. For example, if $C_j = x_1 \vee \neg x_2 \vee \neg x_3$, The pattern Q contains three edges (x_1,α,C_i), (x_2,α,C_i) and (x_3,α,C_i), and the precondition X contains three predicates: (a) $C_j.L_1 = x_1.\mathsf{value}$, (b) $C_j.L_2 \neq x_2.\mathsf{value}$ and (c) $C_j.L_3 \neq x_3.\mathsf{value}$. The consequence of φ_1^1 is $O.\mathsf{sat} = \mathsf{true}$ denoting that $\Psi(\bar{x},\bar{y})$ is true; and (b) φ_2^1 is to lock the application of GARs; more specifically, φ_2^1 is $\emptyset \to O.\mathsf{lock} = \mathsf{true}$, and uses a unary pattern consisting of a node O.
- The set Σ_2 consists of only one GAR φ_1^2 to enforce that $O.\mathsf{sat} = \mathsf{true}$: $O.\mathsf{lock} \neq \mathsf{true} \to O.\mathsf{sat} = \mathsf{true}$, which uses a unary pattern consisting of a node O.

(3) The set P_{out} of *output predicates* is $\{O.\mathsf{sat} = \mathsf{true}\}$.

We next show that $\forall\bar{x}\exists\bar{y}\ \Psi(\bar{x},\bar{y})$ is true if and only if $\mathcal{S}$ is output-commutative for all Γ.

($\Rightarrow$) Assume that $\forall\bar{x}\exists\bar{y}\ \Psi(\bar{x},\bar{y})$ is true. We show that output-commutativity holds for all Γ, *i.e.,* $\mathsf{Chase}(G,\Sigma_1,\mathsf{Chase}(G,\Sigma_2,\Gamma)) \cap P_{\mathsf{out}} = \mathsf{Chase}(G,\Sigma_2,\mathsf{Chase}(G,\Sigma_1,\Gamma)) \cap P_{\mathsf{out}}$. Consider the following two cases.

(1) Assume that $O.\mathsf{lock}{\neq}\mathsf{true}$ does not hold in Γ. We start with $\mathsf{Chase}(G,\Sigma_2,\mathsf{Chase}(G,\Sigma_1,\Gamma))$. Because $\forall\bar{x}\exists\bar{y}\ \Psi(\bar{x},\bar{y})$ is true, for any truth-assignment μ_x for $\bar{x}$, there exists a truth-assignment μ_y such that $\Psi(\bar{x},\bar{y}) = \mathsf{true}$. (I) When Γ encodes a truth assignment for $\bar{x}$, the GAR $\varphi_1^1 \in \Sigma_1$ can be applied, and both predicates $O.\mathsf{sat} = \mathsf{true}$ and $O.\mathsf{lock} = \mathsf{true}$ are added to $\mathsf{Chase}(G,\Sigma_1,\Gamma)$. Because $O.\mathsf{sat} = \mathsf{true}$ already exists in $\mathsf{Chase}(G,\Sigma_1,\Gamma)$, the unique GAR in Σ_2 cannot be applied. That is, $\mathsf{Chase}(G,\Sigma_2,\mathsf{Chase}(G,\Sigma_1,\Gamma)) \cap P_{\mathsf{out}} = \{O.\mathsf{sat} = \mathsf{true}\}$. For $\mathsf{Chase}(G,\Sigma_1,\mathsf{Chase}(G,\Sigma_2,\Gamma))$. Because $O.\mathsf{lock} \neq \mathsf{true}$ does not hold in Γ, the unique GAR in Σ_2 cannot be applied, *i.e.,* $\mathsf{Chase}(G,\Sigma_2,\Gamma) = \emptyset$. Then only GARs in Σ_1 can be applied, and $\mathsf{Chase}(G,\Sigma_1,\mathsf{Chase}(G,\Sigma_2,\Gamma)) \cap P_{\mathsf{out}} = \mathsf{Chase}(G,\Sigma_2,\mathsf{Chase}(G,\Sigma_1,\Gamma)) \cap P_{\mathsf{out}}$. (II) When Γ does not encode a truth assignment for $\bar{x}$, only $O.\mathsf{lock} = \mathsf{true}$ can be deduced by φ_2^1 in Σ_1. Then $\mathsf{Chase}(G,\Sigma_1,\mathsf{Chase}(G,\Sigma_2,\Gamma)) \cap P_{\mathsf{out}} = \mathsf{Chase}(G,\Sigma_2,\mathsf{Chase}(G,\Sigma_1,\Gamma)) \cap P_{\mathsf{out}} = \emptyset$; *i.e.,* $\mathcal{S}$ is output-commutativity.

(2) Assume that $O.\mathsf{lock} \neq \mathsf{true}$ holds in Γ. We show that both sets are $\{O.\mathsf{sat} = \mathsf{true}\}$. First consider $\mathsf{Chase}(G,\Sigma_1,\mathsf{Chase}(G,\Sigma_2,\Gamma))$. Because $O.\mathsf{lock} \neq \mathsf{true}$ holds in Γ, $O.\mathsf{sat} = \mathsf{true}$ is deduced by φ_1^2 in Σ_2. Then $\mathsf{Chase}(G,\Sigma_1,\mathsf{Chase}(G,\Sigma_2,\Gamma)) \cap P_{\mathsf{out}} = \{O.\mathsf{sat} = \mathsf{true}\}$. Next consider $\mathsf{Chase}(G,\Sigma_2,\mathsf{Chase}(G,\Sigma_1,\Gamma))$. Similar to the analysis for case (1), both predicates $O.\mathsf{sat} = \mathsf{true}$ and $O.\mathsf{check} = \mathsf{true}$ are added to $\mathsf{Chase}(G,\Sigma_1,\Gamma)$. Therefore, $\mathsf{Chase}(G,\Sigma_1,\mathsf{Chase}(G,\Sigma_2,\Gamma)) \cap P_{\mathsf{out}}{=}\mathsf{Chase}(G,\Sigma_2,\mathsf{Chase}(G,\Sigma_1,\Gamma)) \cap P_{\mathsf{out}} = \{O.\mathsf{sat} = \mathsf{true}\}$; *i.e.,* $\mathcal{S}$ is output-commutativity.

($\Leftarrow$) Assume that $\mathcal{S}$ is output-commutativity. We show by contradiction that $\forall\bar{x}\exists\bar{y}\ \Psi(\bar{x},\bar{y})$ is true. When $\forall\bar{x}\exists\bar{y}\ \Psi(\bar{x},\bar{y}) = \mathsf{false}$, there exists a truth-assignment μ_x for $\bar{x}$ such that for any truth-assignment μ_y, $\Psi(\bar{x},\bar{y})$ is false. Based on μ, we construct a set Γ_x of validated facts as follows: (1) if variable $\mu_x(x_i) = \mathsf{true}$, then Γ_x contains a fact $v_i.\mathsf{value} = \mathsf{true}$; otherwise it contains $v_i.\mathsf{value} = \mathsf{false}$; (2) $Y_T.\mathsf{value} = \mathsf{true}$ and $Y_F.\mathsf{value} = \mathsf{true}$; (3) $O.\mathsf{lock} = \mathsf{false}$; and (4) for the seven vertices labeled

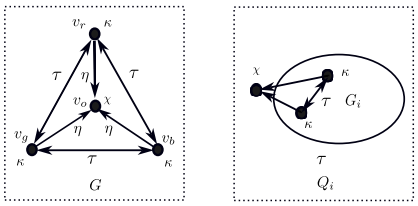


Fig. 7: Graph and patterns in the reduction

C, it encodes the seven satisfying truth-assignments for each clause, *i.e.,* (a) $C_1.L_1 = 0, C_1.L_2 = 0, C_1.L_3 = 1$, ... and (g) $C_1.L_1 = 1, C_1.L_2 = 1, C_1.L_3 = 1$. We next show that $\mathsf{Chase}(G, \Sigma_1, \mathsf{Chase}(G, \Sigma_2, \Gamma)) \cap P_{\mathsf{out}} \neq \mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma)) \cap P_{\mathsf{out}}$. Indeed, because $\forall \bar{x} \exists \bar{y}\ \ \Psi(\bar{x}, \bar{y})$ is false, the GAR φ_1^1 in Σ_1 cannot be applied, and $\mathsf{Chase}(G, \Sigma_1, \Gamma)$ contains only $O.\mathsf{lock}{=}\mathsf{true}$, which prevents the application of the unique GAR φ_1^2 in Σ_2. Then $\mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma)) \cap P_{\mathsf{out}} = \emptyset$. On the other hand, $\mathsf{Chase}(G, \Sigma_2, \Gamma) = \{O.\mathsf{sat} = \mathsf{true}\}$, since $O.\mathsf{lock} \neq \mathsf{true}$ (*i.e.,* $O.\mathsf{lock} = \mathsf{false}$) holds in Γ. Then no mater whether GARs in Σ_1 can be applied, $\mathsf{Chase}(G, \Sigma_1, \mathsf{Chase}(G, \Sigma_2, \Gamma)) \cup P_{\mathsf{out}} = \{O.\mathsf{sat} = \mathsf{true}\}$. Therefore, $\mathsf{Chase}(G, \Sigma_1, \mathsf{Chase}(G, \Sigma_2, \Gamma)) \cap P_{\mathsf{out}} \neq \mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma)) \cap P_{\mathsf{out}}$, *i.e.,* $\mathcal{S}$ is not output-commutativity, a contradiction.

Data complexity. We next show the data complexity, *i.e.,* the problem is $\mathsf{P}_{||}^{\mathsf{NP}}$-complete when the validated fact set Γ is fixed.

Upper bound. Consider the complement problem: there exist $\Sigma_i, \Sigma_j \in \mathcal{S}$ and a fact set Γ such that the two compositions disagree on P_{out}. A certificate consists of $(\Sigma_i, \Sigma_j, \Gamma, f)$, where $f \in P_{\mathsf{out}}$ is a distinguishing output fact. It suffices to verify that $f \in \mathsf{Chase}(G, \Sigma_i, \mathsf{Chase}(G, \Sigma_j, \Gamma))$ but $f \notin \mathsf{Chase}(G, \Sigma_j, \mathsf{Chase}(G, \Sigma_i, \Gamma))$. For each predicate $f \in P_{\mathsf{out}}$, the membership test can be decided using an NP oracle for the verification problem of GARs [3]. Since there exist at most polynomial many such predicates in P_{out} (see the proof of Theorem 1), the check is in $\mathsf{P}_{||}^{\mathsf{NP}}$ by conducting polynomial many NP oracle queries for the membership test. Hence, the original universal property is in $\mathsf{P}_{||}^{\mathsf{NP}}$.

Lower bound. We prove the problem is $\mathsf{P}_{||}^{\mathsf{NP}}$-hard by reduction from the odd max true 3SAT (OMT3) problem, which is $\mathsf{P}_{||}^{\mathsf{NP}}$-complete [15], [14]. The OMT3 problem is to decide, given a 3SAT formula Φ with variables $\bar{x}$, if $\mu_{\max}(\bar{x})$ is the satisfying truth-assignment with maximum true among all assignments, whether $\mu_{\max}(\bar{x})$ sets an odd number of variables true. Denote by $|\mu_{\max}(\bar{x})|$ the number of variables being true in $\mu_{\max}(\bar{x})$. The problem remains $\mathsf{P}_{||}^{\mathsf{NP}}$-complete even under the assumption that $|\mu_{\max}(\bar{x})| \geq 1$; indeed, it is in PTIME to check whether Φ is satisfied by setting all variables to be false.

Given a 3SAT formula Φ, we construct G, $\mathcal{S}$ and Γ such that $\mu_{\max}(\bar{x})$ sets an odd number of variables true if and only if output-commutativity holds for $\mathcal{S}$. Intuitively, we use G to encode all possible truth-assignments for variables in $\bar{x}$, $\mathcal{S} = \{\Sigma_1, \Sigma_2\}$ to encode the structure of Φ, and a predicate p_0 to indicate whether $\mu_{\max}(\bar{x})$ sets an odd number of variables true.

The reduction takes steps through the problems as follows.

(1) We first consider a variant of the OMT3 problem, denoted by (Φ, k), which is to decide whether all satisfying truth-assignments for Φ set less than k variables true, *i.e.,* $|\mu_{\max}(\bar{x})| \leq k$. The problem is in coNP [14]. Assume that Φ has n variables. Consider the following $n+1$ instances $(\Phi, 0), \ldots, (\Phi, n)$ of the variant. We can verify that $|\mu_{\max}(\bar{x})|$ is odd if and only if there exists an odd number k such that (Φ, k) returns true but $(\Phi, k+1)$ returns false.

(2) We next encode an instance (Φ, k) by a 3-colorability problem, from which we construct G, $\mathcal{S}$ and Γ. The 3-colorability problem is to decide, given a graph G whether there exists a 3-coloring μ of G such that for each edge (u, v) in G, $\mu(u) \neq \mu(v)$, which is known to be NP-complete [12]. Given an instance (Φ, k) we construct a graph G_k such that G_k is 3-colorable if and only (Φ, k) returns false. Let $G_0, \ldots, G_n$ be the graph constructed from $(\Phi, 0), \ldots (\Phi, n)$, respectively.

(3) Given $G_0, \ldots, G_n$, we construct G, $\mathcal{S}$ and Γ as follows: (a) G consists of a clique with three vertices representing all possible coloring, and vertex v_o to indicate whether $\mathcal{S}$ is output-commutative; (b) $\mathcal{S}$ consists of two sets Σ_1 and Σ_2, where Σ_1 encodes topological structures of $G_0, \ldots, G_n$, and Σ_2 calculates how many graphs is 3-colorable; and (c) Γ encodes the valid coloring of the three vertices in G.

More specifically, G, $\mathcal{S}$ and Γ are constructed as follows.

(1) The graph $G = (V, E, \mathcal{L}, \ell, \mathcal{A}, \lambda)$ is shown in Figures 7, where (a) $V = \{v_r, v_g, v_b, v_o\}$; vertices v_r, v_g, v_b encode the 3 colors, and v_o indicates whether $\mathcal{S}$ is output-commutative; (b) $E = E_c \cup E_o$ with $E_c = \{(v_r, \tau, v_g), (v_r, \tau, v_b), (v_r, \tau, v_r), (v_g, \tau, v_b), (v_b, \tau, v_r), (v_b, \tau, v_g)\}$ and $E_o = \{(v_r, \eta, v_o), (v_g, \eta, v_o), (v_b, \eta, v_o)\}$, where E_c forms a clique and E_o links three vertices v_r, v_g, v_b to the center v_o; (c) $\ell(v_r) = \ell(v_g) = \ell(v_b) = \kappa$ and $\ell(v_o) = \chi$; and (d) $\lambda(v_r).C = r$, $\lambda(v_g).C = g$ and $\lambda(v_b).C = b$, *i.e.,* the attributes C of v_r, v_g and v_b are three different colors r, g and b, respectively.

(2) The set Σ_1 consists of $n+1$ GARs, one for each graph G_i ($i \in [0, n]$) constructed above. Given G_i, we construct a GAR $\varphi_i = Q_i[\bar{x}](X_i \rightarrow p_0)$, where

(A) the pattern Q_i is almost the same as G_i (see Figures 7), except that (a) Q_i has an additional vertex v_o to indicate whether G_i is 3-colorable; (b) each undirected edge (u, v) in G is represented by two directed edges (u, τ, v) and (v, τ, u); and (c) each vertex v in G_i has an edge (v, η, v_o) leading to v_o;

(B) precondition X_i contains a predicate $u.C \neq v.C$ for each edge (u, τ, v), *i.e.,* adjacent vertices carry different colors; and

(C) consequence p_o is $v_o.C_i = \mathsf{true}$. Here we use different attributes to denote different graphs, one for each of $G_0, \ldots, G_n$.

(3) Set $\Sigma_2 = \{\varphi_v\}$, where $\varphi_v = Q[\bar{x}](\mathcal{M}(v_o) \rightarrow v_o.\mathsf{Valid} = \mathsf{true})$, $Q[\bar{x}]$ consists of only one vertex v_o labeled χ, and the model $\mathcal{M}(v_o)$ checks whether vertex v_o has a positive even number of attributes C_i ($i \in [1, n]$) satisfying $v_o.C_i = \mathsf{true}$.

(4) The set Γ of *validated facts* consists of the following facts: $\lambda(v_r).C = r$, $\lambda(v_g).C = g$ and $\lambda(v_b).C = b$, *i.e.,* the attributes

C of v_r, v_g and v_b encodes colors r, g and b, respectively.

(5) The set $\mathcal{P}_{\mathsf{out}}$ of *output predicates* is $\{v_o.\mathsf{Valid} = \mathsf{true}\}$.

We next show that $\mu(\bar{x})$ sets an odd number of variables true if and only if output-commutativity holds for all $\mathcal{S}$.

($\Rightarrow$) Assume that $\mu_{\max}(\bar{x})$ sets an odd number of variables true. We next show that output-commutativity holds for all Γ, *i.e.,* $\mathsf{Chase}(G, \Sigma_1, \mathsf{Chase}(G, \Sigma_2, \Gamma)) \cap P_{\mathsf{out}} = \mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma)) \cap P_{\mathsf{out}}$. Consider when applying GARs in Σ_2 first, $\mathsf{Chase}(G, \Sigma_2, \Gamma) = \emptyset$; indeed, since no predicate $v_o.C_i = \mathsf{true}$ ($i \in [0, n]$) exists in Γ, GAR $\varphi_v \in \Sigma_2$ cannot be applied. After applying Σ_1, $\mathsf{Chase}(G, \Sigma_1, \mathsf{Chase}(G, \Sigma_2, \Gamma)) \cap P_{\mathsf{out}} = \emptyset$. More specifically, when first applying Σ_1, consider the following two cases: (1) none of GARs in Σ_1 can be applied, then $\mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma)) \cap P_{\mathsf{out}} = \emptyset$, and the output-commutativity holds. (2) Otherwise, when applying GARs in Σ_1, $\mathsf{Chase}(G, \Sigma_1, \Gamma)$ consists of $v_o.C_i = \mathsf{true}$ for each $i \in [1, T]$. But $\mu_{\max}(\bar{x})$ sets an odd number T of variables true, GAR $\varphi_v \in \Sigma_2$ cannot be applied, and predicate $v_o.\mathsf{Valid} = \mathsf{true}$ is not in $\mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma))$. Therefore, $\mathsf{Chase}(G, \Sigma_1, \mathsf{Chase}(G, \Sigma_2, \Gamma)) \cap P_{\mathsf{out}} = \mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma)) \cap P_{\mathsf{out}} = \emptyset$; $\mathcal{S}$ has output-commutativity.

($\Leftarrow$) Conversely, assume that $\mathcal{S}$ has output-commutativity. We show by contradiction that $\mu_{\max}(\bar{x})$ sets an odd number of variables true. Assume by contradiction that $|\mu_{\max}(\bar{x})|$ is even. We show that the output-commutativity does not hold. That is, $\mathsf{Chase}(G, \Sigma_1, \mathsf{Chase}(G, \Sigma_2, \Gamma)) \cap P_{\mathsf{out}} \neq \mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma)) \cap P_{\mathsf{out}}$. Indeed, $\mathsf{Chase}(G, \Sigma_1, \Gamma)$ consists of $v_o.C_i = \mathsf{true}$ for each $i \in [1, |\mu_{\max}(\bar{x})|]$. Because $|\mu_{\max}(\bar{x})|$ is even, the unique GAR $\varphi_v \in \Sigma_2$ can be applied, and the predicate $v_o.\mathsf{Valid} = \mathsf{true}$ is added to $\mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma))$. On the other hand, $\mathsf{Chase}(G, \Sigma_2, \Gamma)) = \emptyset$, since no predicate $v_o.C_i = \mathsf{true}$ ($i \in [0, n]$) exists in Γ. Then GAR $\varphi_v \in \Sigma_2$ cannot be applied, and predicate $v_o.\mathsf{Valid} = \mathsf{true}$ is not in $\mathsf{Chase}(G, \Sigma_1, \mathsf{Chase}(G, \Sigma_2, \Gamma))$. Therefore, $\mathsf{Chase}(G, \Sigma_1, \mathsf{Chase}(G, \Sigma_2, \Gamma)) \cap P_{\mathsf{out}} \neq \mathsf{Chase}(G, \Sigma_2, \mathsf{Chase}(G, \Sigma_1, \Gamma)) \cap P_{\mathsf{out}}$, *i.e.,* $\mathcal{S}$ has no output-commutativity, a contradiction. □

PROOF OF THEOREM 5

Proof: (*Upper bound*). A certificate is a subset $O' \subseteq O$. We can verify in time polynomial in $|O|$ that O' is conflict-free by checking all pairs in O' against Conf, and that its total weight is at least K by summation. Hence OptCanon is in NP.

Lower bound. For the lower bound, we reduce from the independent set (IS) problem, which is NP-complete (cf. [12]). The IS problem is to decide, given a graph $G = (V, E)$ and a number K_I, whether there exists a subset $V' \subseteq V$ such that $|V'| \geq K_I$ and no two vertices u and v in V' are adjacent in G. Given G and K_I, we construct a set O of candidates, a symmetric conflict relation Conf, a non-negative weight function w, and an integer K_O such that there exists an optimal canonical output set of total weight at least K_O if and only G has an independent set V' with $|V'| \geq K_I$.

Given a graph $G = (V, E)$ and the number K_I, we construct an instance of OptCanon as follows. For each vertex $v \in V$, create a candidate output fact $o_v \in O$ with weight $w(o_v)$=1. Define the conflict relation Conf such that $(o_u, o_v) \in \mathsf{Conf}$ iff $(u, v) \in E$. Then selecting a conflict-free subset $O' \subseteq O$ of total weight at least K_O=K_I corresponds exactly to selecting an independent set of at least K_I in G. Since the reduction uses only the output candidates and their pairwise conflicts, and does not depend on the rule patterns or predicate vocabulary, the hardness holds even when Σ and P_{out} are fixed.

Combining membership and hardness, OptCanon is NP-complete, and the optimization version is NP-hard. □